

卷积神经网络实现报告

第 9 小组 王肖鉴，冯越，方俊皓，梁启宸

June 2, 2025

1 引言

1.1 卷积的数学原理

卷积是一种数学上由两个函数生成第三个函数的运算，其本质是滑动加权求和，可视为一种特殊的积分操作。在信号处理、图像处理与系统建模等领域中，卷积被广泛用于滤波与特征提取。

二维离散卷积的数学表达式为：

$$Y(x, y) = \sum_m \sum_n X(m, n) H(x - m, y - n)$$

该定义表示对输入信号 X 与滤波核 H 进行加权累加，以捕捉局部结构与空间相关性。在频域中，卷积对应傅里叶变换下的点乘操作：

$$Y_{\text{Freq}} = X_{\text{Freq}} \cdot H_{\text{Freq}}$$

这体现了卷积的本质为频率选择性滤波器。

1.2 卷积在深度神经网络中的应用

卷积神经网络（Convolutional Neural Network, CNN）将卷积机制引入图像分类、语义分割等端到端任务中。需要注意的是，CNN 中通常使用的是互相关（cross-correlation）操作，并未对卷积核进行翻转，其形式与标准卷积不同，但通过训练可以学得与目标任务匹配的最优核方向，等效翻转操作。

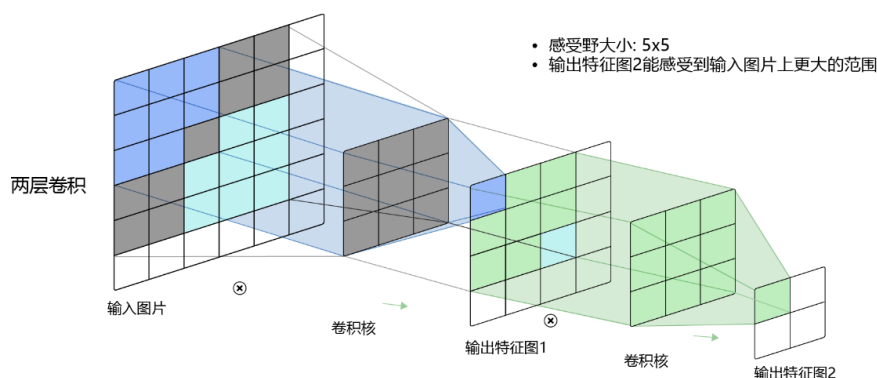


Figure 1: CNN 卷积示意图

CNN 中的卷积层通过滑动卷积核，对局部感受野内的像素进行加权求和，输出特征图。每一个卷积核负责提取一种局部特征（如边缘、角点、纹理等）。其空间输出维度满足如下关系：

$$\text{out} = \left\lfloor \frac{\text{in} + 2 \cdot \text{padding} - \text{kernel_size}}{\text{stride}} \right\rfloor + 1$$

1.3 实验任务与目标

本报告围绕卷积神经网络的基本原理和实际应用，主要完成了以下几个方面的任务：

- 手动实现二维卷积操作，并尝试用向量化方法提升效率；
- 搭建并训练了一个简单的卷积神经网络（LeNet），用于手写数字识别；
- 对模型中的卷积核和中间特征进行了可视化，帮助理解网络如何提取图像特征；
- 模型卷积核与中间特征响应的可视化分析；
- 卷积维度拓展至 1D 和 3D 的实现及应用验证；
- 卷积实现的性能测试与分析；
- 自定义卷积核的设计与任务效果评估。

通过这些实验，我们不仅掌握了卷积的数学原理和代码实现，也直观体会到卷积在文本、图像、视频等领域的广泛用途和强大能力。

2 基础任务

2.1 二维卷积操作的手动实现

2.1.1 任务问题

本任务要求手动实现支持多通道输入输出、可配置 padding 与 stride、支持多组卷积核的二维卷积操作，需实现二维信息基础特征提取，并可视化卷积响应。

2.1.2 原理分析

卷积运算是一种通过局部加权求和实现特征提取的数学运算，其核心在于滑动窗口机制。在深度学习框架中，卷积层通过卷积核（kernel/filter）在输入矩阵上以滑动窗口方式遍历每一个位置，对每个局部区域与卷积核进行元素对应相乘并求和，将局部信息映射为高层特征。多通道输入输出用于处理如 RGB 图像等多维特征数据，stride 用于控制卷积核的滑动步长以优化模型和减少计算量，padding 用于在步长变化时保留边缘信息，多组不同的卷积核则实现多样化的特征提取。卷积的数学公式为：

$$Y[i, j] = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} X[i-m, j-n] \cdot W[m, n] + b \quad (1)$$

但是，由于我们使用的卷积核是通过计算机训练出来的，在这一步的操作中实际已经包含了旋转操作，所以我们在实现时可以直接使用互相关运算而不需要进行旋转，于是我们参考如下公式实现：

$$Y[i, j] = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} X[i+m, j+n] \cdot W[m, n] + b \quad (2)$$

从最原始的数学公式出发，输入数据经过填充 (padding) 扩展边界后，卷积核以固定步长 (stride) 在平面滑动，每次覆盖一个与核尺寸相同的局部区域 (如 3×3)，通过对应相乘累加运算生成输出特征图的对应位置值。这一过程中，每个输出位置的数值来源于输入局部区域与核参数的加权求和，而偏置项 b 的设置则是为了给每个输出通道引入可学习的偏移量，增强模型表达能力。

2.1.3 算法设计与实现

1. 普通方法

具体实现时，输入数据（如图像）通常表示为四维张量（批量大小、输入通道数、高度、宽度）。核参数以四维形式存在（输出通道数、输入通道数、核高、核宽），反映了多通道交互的特性：每个输出通道的核独立学习对输入所有通道的局部特征提取方式。例如，RGB 图像输入（3 通道）经过卷积生成 2 通道输出时，每个输出通道的核会对 3 个输入通道分别进行二维卷积，再将三个通道的结果相加，最后叠加偏置。这种设计既保留了通道间的特征融合能力，又通过参数共享（同一核在空间维度滑动）大幅减少了参数量。

传统实现采用四重嵌套循环（遍历批次、输出通道、输出高度、输出宽度），直观地模拟了卷积的物理过程：对于每个输出位置，卷积核与输入的感受野区域逐通道进行乘积累加。例如，当步长为 1 且无填充时， 5×5 输入经 3×3 核卷积后输出 3×3 特征图，核的每次滑动仅覆盖输入的一个局部窗口，通过窗口内元素与核参数的对应相乘之和完成特征提取。填充操作通过补零控制输出尺寸，避免边缘信息丢失；步长则决定核滑动的间隔，步长越大，输出尺寸越小，计算量越低。

值得注意的是，数学意义上的卷积需将核旋转 180 度后再进行滑动计算，但深度学习框架通常省略旋转步骤（等价于直接进行互相关运算），因为核参数本身通过反向传播学习，旋转操作已被参数优化过程隐式包含。这种设计简化了计算流程，同时保证了特征提取的有效性。卷积的局部连接特性使其天然具备平移不变性，适用于图像、语音等具有空间或时序结构的数据处理。

2. 向量化优化

但是，卷积的传统实现通过四层嵌套循环逐点计算局部感受野与核的加权求和，其时间复杂度随输入尺寸、通道数和核大小增大而快速增长。这种逐元素计算模式不仅无法利用现代硬件的并行能力，还会因为内存访问碎片化造成缓存利用率低下，严重影响深度学习模型的训练与推理效率，所以需要对其进行优化。

我们注意到，向量化的运算可以提升运行的效率。向量化实现通过滑动窗口将输入展开 (unfold) 为矩阵结构（行向量组），利用内存连续性调整 (permute+view) 将四维输入张量重组为 (batch, in_channel, kh*kw) 的三维张量，同时将四维核权重展平为 (out_channel, in_channel, kh*kw) 的三维张量，最终通过单次批量矩阵乘法（广播运算）完成所有输出的计算。

这种变换的优势在于将卷积操作转化为高度并行的矩阵运算，能够充分利用 GPU/CPU 的并行计算能力，极大提升计算效率。此外，向量化展开后的数据在内存中呈现空间连续分布，减少了循环实现中频繁的非连续内存访问，从而显著提高了缓存命中率和整体运行速度。总体而言，向量化实现不仅优化了计算性能，也提升了模型在大规模数据处理下的实际应用能力。

3. unfold 实现

对于 unfold 的实现，我们使用 PyTorch 的 `torch.Tensor.unfold` 方法提取滑动窗口，生成一个三维张量，其中每个窗口对应一个输出位置。接着，通过矩阵乘法将所有窗口与卷积核进行批量计算，最后加上偏置项得到最终输出。这种方法不仅提高了计算效率，还使得代码更简洁易读。具体实现原理如下图所示：

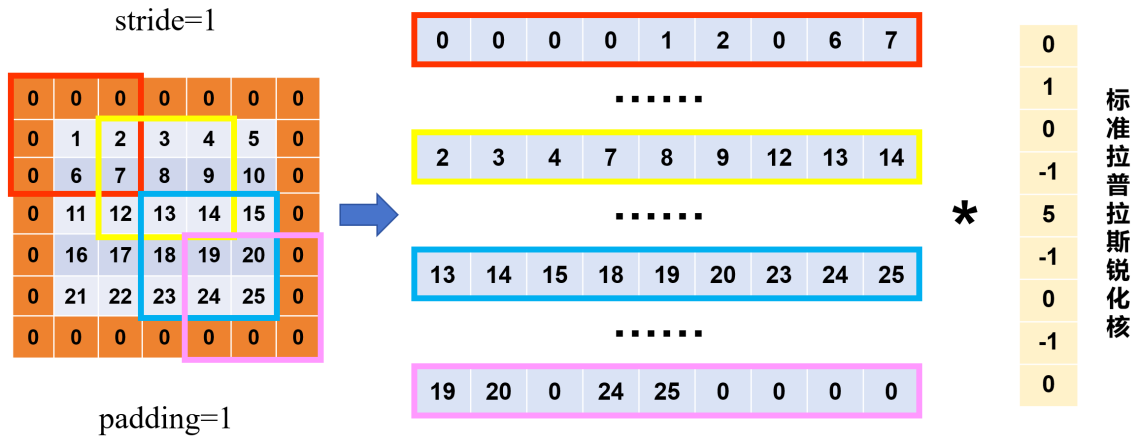


Figure 2: unfold 实现原理图

unfold 操作的核心在于将输入张量的空间维度展开为多个局部窗口，每个窗口对应卷积核的感受野。通过这种方式，我们可以将卷积操作的对应相乘转化为矩阵乘法，从而利用高效的线性代数库进行加速（广播操作）。

4. 算法实现

```

1 def forward_vectorized(self, x):
2     # 输入维度校验
3     if x.dim() != 4:
4         raise ValueError(" 输入张量必须为 4 维 (batch, channel, height, width)")
5     if x.size(1) != self.in_channel:
6         raise ValueError(
7             f" 输入通道数不匹配: 期望 {self.in_channel} 个通道, 实际输入 {x.size(1)} 个通道"
8         )
9
10    batch_size, in_channels, hin, win = x.shape
11    kh, kw = self.kernel_size
12    hout = (hin + 2*self.padding - kh) // self.stride + 1
13    wout = (win + 2*self.padding - kw) // self.stride + 1
14
15    # 执行填充
16    x_padded = nn.functional.pad(
17        x,
18        (self.padding, self.padding, self.padding, self.padding),
19        mode='constant',
20        value=0
21    )

```



```

22     # 使用 unfold 提取滑动窗口
23     x_unfold = x_padded.unfold(2, kh, self.stride).unfold(3, kw, self.stride) # 形状:
        ↳ (batch, in_ch, hout, wout, kh, kw)
24     # 维度调整和展平
25     x_unfold = x_unfold.permute(0, 2, 3, 1, 4, 5).contiguous() # 调整为
        ↳ (batch, hout, wout, in_ch, kh, kw)
26     x_unfold = x_unfold.view(batch_size, hout * wout, -1)
27
28     weight_matrix = self.weight.view(self.out_channel, -1) # 展平权重
29     # 矩阵乘法计算卷积结果
30     output = torch.matmul(x_unfold, weight_matrix.t()) # 结果形状
        ↳ (batch, hout*wout, out_ch)
31     # 添加偏置
32     output += self.bias.view(1, 1, -1)
33
34     # 最终形状调整 (batch, out_ch, hout, wout)
35     output = output.permute(0, 2, 1).view(batch_size, self.out_channel, hout, wout)
36
37     return output

```

2.1.4 运行结果与分析

1. 多通道输入输出：对于多通道输入输出，我们采用了标准测试样例中提供的三张 RGB 图片作为输入，通过手动实现的卷积操作提取特征，输出如下：

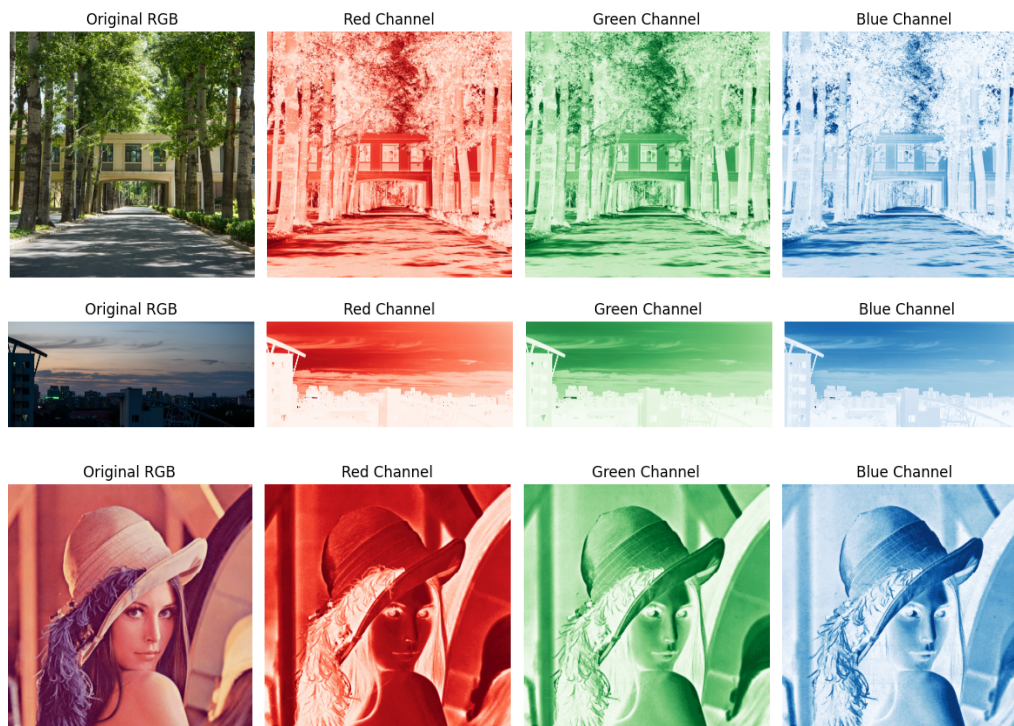


Figure 3: 分别对测试样例 1、2、3 进行多通道输入输出卷积特征提取的结果

2. 步长与边界填充： 对于可调整的 padding 和 stride 参数，我们使用了 `lenna.bmp` 文件作为测试样例，基于高斯模糊卷积核，设置步长分别为 1, 2, 3，得到了以下的输出结果：

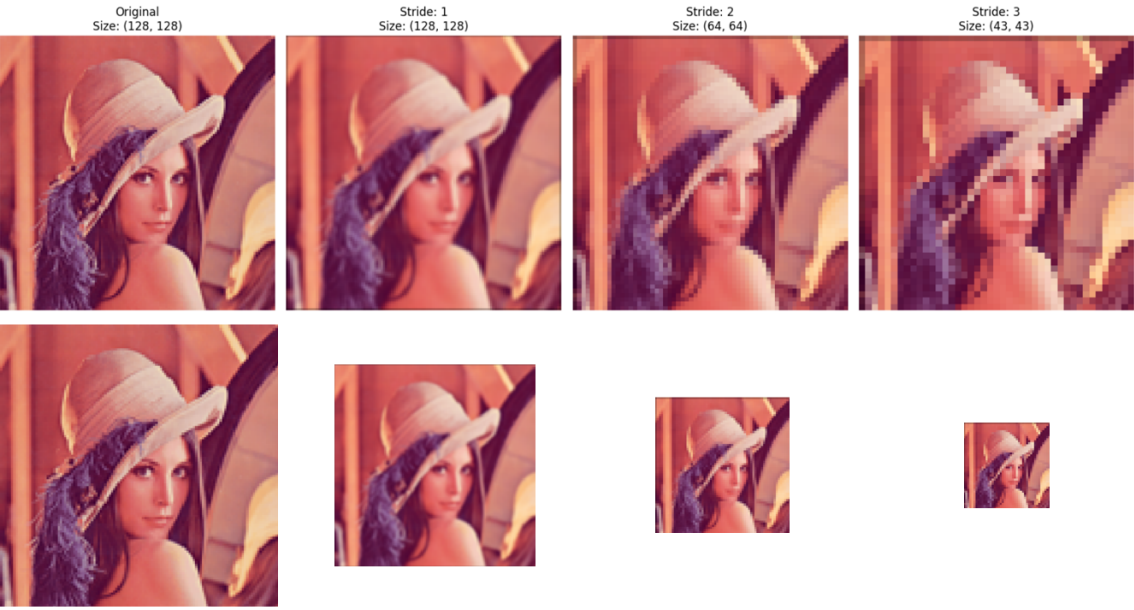


Figure 4: padding & stride 测试结果

可以看到，随着步长的增大，输出特征图的尺寸逐渐减小，且模糊效果也有所增强。这验证了 padding 和 stride 参数对卷积操作的影响。

3. 卷积核运算效果： 为验证卷积核运算的效果，我们使用了多种常见的卷积核对 `lenna.bmp` 图像进行处理，得到以下结果：

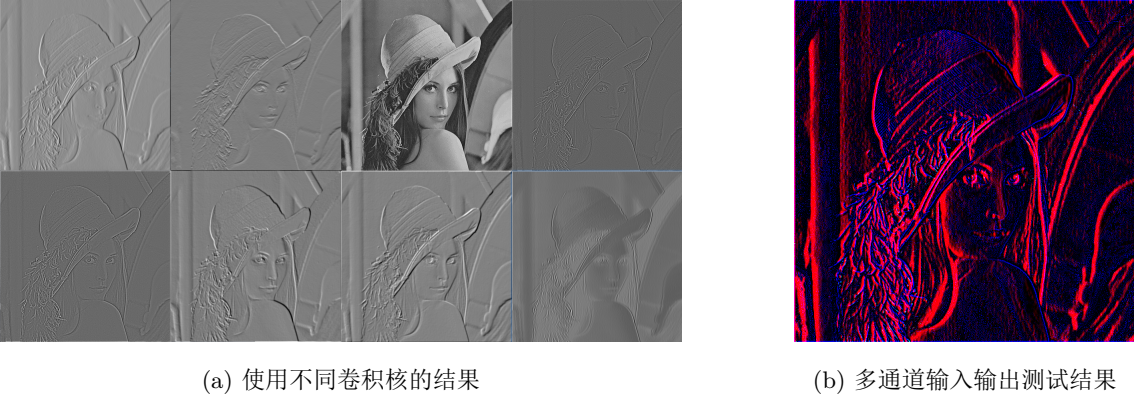


Figure 5: 对 `lenna.bmp` 应用二维卷积测试结果

图 5a 中，第一行从左到右使用的卷积核分别为：`sobel_x`|`sobel_y`|`sharpen`|`laplace`；第二行从左到右使用的卷积核分别为：`highpass`|`TL-BR`|`TR-BL`|`Gabor`。据此实验结果，可以观察到我们手动实现的二维卷积函数能够成功加载并应用多种卷积核，且输出效果符合预期。

图 5b 中，依结果来看 RGB 三通道的卷积融合效果良好，验证了我们手动实现的二维卷积函数具有多通道输入输出能力。

2.1.5 性能测试与分析

我们对手动实现的二维卷积操作进行了性能测试，主要包含正确率测试以及运行效率测试。正确率测试通过与 PyTorch 内置的卷积函数的运行结果进行对比，验证了手动实现的卷积操作在功能上的正确性。运行效率测试则通过对比手动实现的函数与 PyTorch 内置函数的单位时间平均吞吐量来评估了我们手动实现函数的性能。具体实现函数如下：

```
1 class ConvBenchmarker:
2     def __init__(self, tester):
3         self.tester = tester
4         self._setup_models()
5
6     def _setup_models(self):
7         """ 初始化两种卷积模型 """
8         self.model_configs = [
9             ("PyTorch", self._build_model(PyTorchConv2d)), ("Self_Vectorized",
10                 ↪ self._build_model(VectorizedConvWrapper)),
11         ]
12
13     def _build_model(self, conv_class):
14         """ 构建双层卷积模型 """
15         class SafeModel(nn.Module):
16             def __init__(self):
17                 super().__init__()
18                 self.conv1 = conv_class(3, 64, kernel_size=3, padding=1)
19                 self.conv2 = conv_class(64, 128, kernel_size=3, padding=1)
20
21             def forward(self, x):
22                 with torch.no_grad(): # 禁用梯度计算
23                     x = self.conv1(x)
24                     return self.conv2(x)
25
26             def cleanup(self):
27                 """ 显式释放模型资源 """
28                 for p in self.parameters():
29                     if p.grad is not None:
30                         del p.grad
31                 torch.cuda.empty_cache()
32
33         return SafeModel()
34
35     def _warmup(self, model, device, batch_size=4):
36         """ 预热 GPU """
37         dummy_input = torch.randn(batch_size, 3, *self.tester.img_size).to(device)
```

```

37         for _ in range(3):
38             _ = model(dummy_input)
39         del dummy_input
40         torch.cuda.empty_cache()
41
42     def benchmark(self, device='cuda', batch_size=4, test_runs=10):
43         results = []
44         for model_idx, (name, model) in enumerate(self.model_configs):
45             # 模型测试开始提示
46             print(f"\n{'='*40}")
47             print(f" 开始测试模型 [{model_idx+1}/{len(self.model_configs)}]: {name}")
48             print(f" 测试参数: batch_size={batch_size}, test_runs={test_runs}")
49             print(f" 开始时间: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
50
51             try:
52                 model = model.to(device).eval()
53                 self._warmup(model, device, batch_size)
54
55                 time_records = []
56                 data_gen = self.testers.memory_safe_generator(batch_size, device)
57
58                 start_time = datetime.now()
59                 progress = 0
60                 ProgressVisualizer.print_progress(0, test_runs, start_time,
61                 ↪ prefix=" 进度")
62
63                 for run_idx in range(test_runs):
64                     # 自动重置生成器逻辑
65                     try:
66                         batch = next(data_gen)
67                     except StopIteration:
68                         # 重置生成器并重试
69                         data_gen = self.testers.memory_safe_generator(batch_size, device)
70                         batch = next(data_gen)
71
72                     if device.startswith('cuda'):
73                         # CUDA 事件计时 (精确 GPU 耗时)
74                         starter = torch.cuda.Event(enable_timing=True)
75                         ender = torch.cuda.Event(enable_timing=True)
76                         starter.record()
77                         _ = model(batch)
78                         ender.record()
79                         torch.cuda.synchronize()
80                         time_ms = starter.elapsed_time(ender)

```

```

80         time_records.append(time_ms)
81     else:
82         # CPU 计时（高精度时间戳）
83         cpu_start = time.perf_counter() # 修改变量名
84         _ = model(batch)
85         time_ms = (time.perf_counter() - cpu_start) * 1000
86         time_records.append(time_ms)
87
88     # 更新进度
89     progress += 1
90     ProgressVisualizer.print_progress(progress, test_runs, start_time,
91     ↪     prefix=" 进度")
92     # 释放资源
93     del batch
94     model.cleanup()
95     gc.collect()
96
97     avg_time = sum(time_records) / len(time_records)
98     throughput = batch_size * 1000 / avg_time
99     results.append((name, avg_time, throughput))
100
101     ↪     print(f"\n测试完成! 平均耗时: {avg_time:.2f}ms | 吞吐量: {throughput:.2f} img/s")
102
103 except RuntimeError as e:
104     print(f"\n{name} 测试失败: {str(e)}")
105     results.append((name, float('inf'), 0))
106
107 finally:
108     del model
109     torch.cuda.empty_cache()
110     gc.collect()
111
112 return results
113
114 # 卷积包装器定义
115
116 class TraditionalConvWrapper(Conv_2d):
117     def forward(self, x):
118         return self.forward_tradition(x)
119
120 class VectorizedConvWrapper(Conv_2d):
121     def forward(self, x):
122         return self.forward_vectorized(x)
123
124 class PyTorchConv2d(nn.Conv2d):
125     def __init__(self, in_channel, out_channel, kernel_size, stride=1, padding=0):

```

```

122         super().__init__(in_channel, out_channel, kernel_size, stride, padding)

class Conv2dTester:
    def __init__(self):
        self.test_cases = [
            # (input_size, in_channels, out_channels, kernel_size, stride, padding)
            (5, 3, 2, 3, 1, 0),      # 基础测试
            (5, 3, 2, 3, 1, 1),      # 带 padding
            (7, 3, 2, 3, 2, 0),      # 不同步长
            (1, 1, 1, 1, 1, 0),      # 最小输入尺寸
            (7, 3, 2, 5, 1, 2),      # 大尺寸核
            (5, 4, 6, 3, 1, 1),      # 多通道
        ]
        self.results = []

    def _create_conv_layers(self, in_ch, out_ch, kernel_size, stride, padding):
        # 官方实现
        official_conv = nn.Conv2d(in_ch, out_ch, kernel_size,
                                   stride=stride, padding=padding, bias=True)

        # 自定义实现
        custom_conv = Conv_2d(
            in_channel=in_ch,
            out_channel=out_ch,
            kernel_size=kernel_size,
            stride=stride,
            padding=padding,
            weight=official_conv.weight.data.clone(),
            bias=official_conv.bias.data.clone()
        )
        return official_conv, custom_conv

    def _run_single_test(self, case):
        """ 执行单个测试用例并记录误差 """
        input_size, in_ch, out_ch, k, s, p = case
        case_id = f"in{in_ch}_out{out_ch}_k{k}_s{s}_p{p}"

        # 生成随机输入
        x = torch.randn(2, in_ch, input_size, input_size)

        # 创建卷积层
        official_conv, custom_conv = self._create_conv_layers(in_ch, out_ch, k, s, p)

        # 前向传播

```

```

43     official_out = official_conv(x)
44     custom_out = custom_conv(x)
45
46     # 形状检查
47     if official_out.shape != custom_out.shape:
48         print(f"[形状异常] 测试用例 {case_id}")
49         self.results.append({
50             "case": case_id,
51             "max_diff": None,
52             "mean_diff": None,
53             "shape_match": False
54         })
55         return
56
57     # 数值误差计算
58     max_diff = torch.max(torch.abs(official_out - custom_out)).item()
59     mean_diff = torch.mean(torch.abs(official_out - custom_out)).item()
60
61     # 存储结果
62     self.results.append({
63         "case": case_id,
64         "max_diff": max_diff,
65         "mean_diff": mean_diff,
66         "shape_match": True
67     })
68
69     def _generate_labels(self):
70         """ 生成测试用例标签 """
71         labels = []
72         for case in self.test_cases:
73             input_size, in_ch, out_ch, k, s, p = case
74             labels.append(
75                 f"in{in_ch}→out{out_ch}\n"
76                 f"k={k} s={s} p={p}\n"
77                 f"in_size={input_size}"
78             )
79         return labels

```

测试结果如下所示:

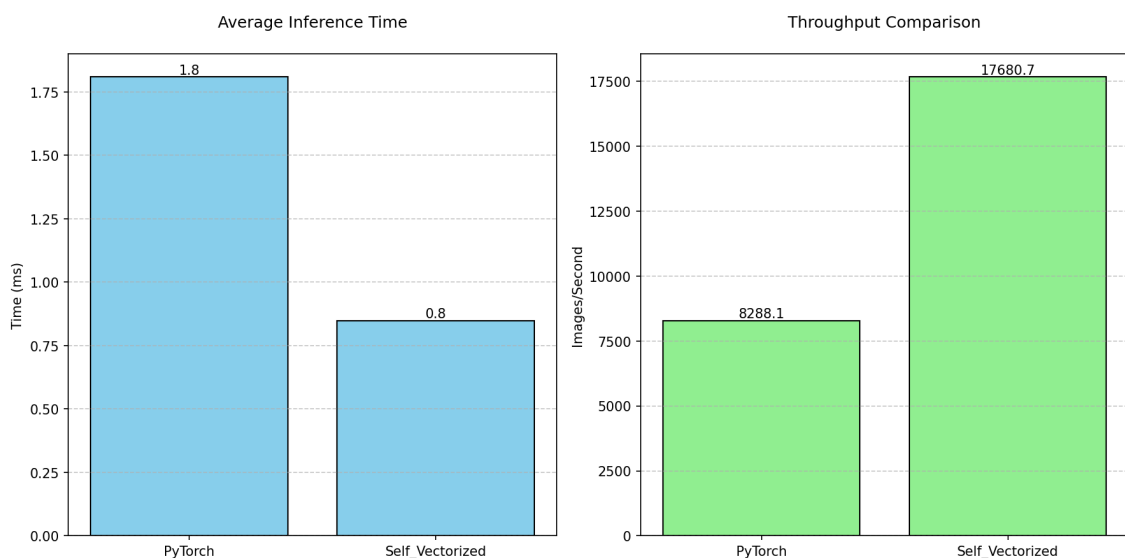


Figure 6: 卷积操作性能测试结果（time）

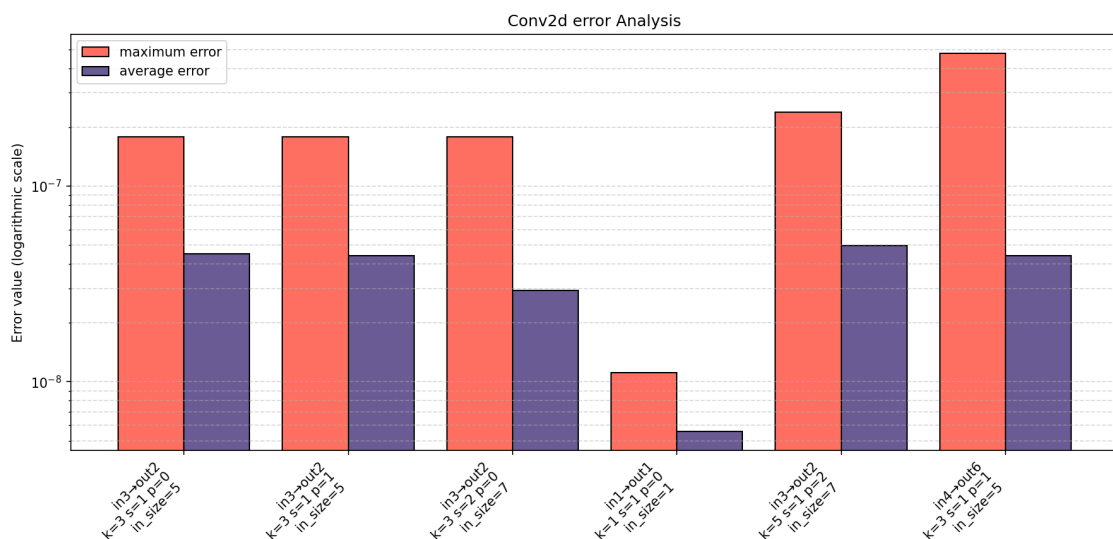


Figure 7: 卷积操作性能测试结果（accuracy）

可以看到整体的运行效率大约是 PyTorch 内置卷积函数的 2 ~ 3 倍左右. 这表明手动实现的卷积操作在效率上有了一定的提升。同时，正确性方面，通过与 PyTorch 内置卷积函数的运行结果对比，可以发现误差均在 $10^{-8} \sim 10^{-7}$ 数量级，这是可以接受的。以上测试结果表明手动实现的卷积操作在功能和性能上都达到了预期目标。

性能提升的具体原因将在 第 3.3.3 节 “性能对比及分析” -2. 性能特征及优化原理分析 中详细讨论。

2.2 小型卷积神经网络的构建与训练

2.2.1 任务问题

本任务要求构建类似 LeNet 的 5 层网络，手动实现所有前向传播层，在 MNIST 数据集达到 95%+ 测试准确率，并分析训练所得卷积核特征。

2.2.2 原理分析

1. 功能模块

卷积神经网络（CNN）由多个功能模块组成，各层在提取特征、压缩空间信息及执行分类任务中扮演不同角色。以下介绍手动实现的功能模块。

卷积层（Convolution Layer） 卷积层通过局部感受野在输入图像上滑动卷积核，提取空间结构特征。卷积操作可表示为：

$$y_{i,j}^{(k)} = \sum_{c=1}^{C_{in}} \sum_{m=1}^K \sum_{n=1}^K x_{i+m,j+n}^{(c)} \cdot w_{m,n}^{(k,c)} + b^{(k)}$$

其中 x 为输入特征图， w 为卷积核， b 为偏置， k 表示输出通道， C_{in} 为输入通道数。该层学习滤波器参数以自动提取边缘、角点、纹理等底层特征。

非线性激活（ReLU） ReLU（Rectified Linear Unit）为最常用的非线性函数，定义为：

$$f(x) = \max(0, x)$$

ReLU 可引入非线性，提高网络表达能力，并缓解梯度消失问题。

池化层（Pooling Layer） 池化层用于降低空间维度、压缩特征图大小，增强模型的平移不变性。最大池化的表达式为：

$$y_{i,j}^{(k)} = \max_{(m,n) \in \mathcal{R}} x_{s \cdot i + m, s \cdot j + n}^{(k)}$$

其中 $\mathcal{R}$ 是池化窗口， s 为步长。该操作保留最显著特征，减少参数数量和计算量。

全连接层（Fully Connected Layer） 全连接层用于特征整合与分类，每个神经元与上一层所有输出连接。其前向传播公式为：

$$y = Wx + b$$

其中 x 是输入特征向量， W 和 b 分别为权重矩阵和偏置项。该层将卷积模块提取的高级特征映射到类别概率空间。

上述模块组合构成了端到端的图像分类器，通过反向传播训练卷积核和权重，使得整个网络能够自动学习判别性特征。

2. 网络结构

本项目实现的 CNN 网络结构基于经典 LeNet-5 架构，包含两个卷积层、两个池化层、三个全连接层，整体结构如下：

- **输入层：** MNIST 图像为 28×28 的单通道灰度图。
- **第 1 卷积层（Conv1）：** 输入通道数为 1，输出通道数为 6，卷积核大小为 5×5 ，padding=2。输出特征图尺寸为 28×28 。
- **第 1 池化层（Pool1）：** 采用 2×2 的最大池化，步长为 2。特征图尺寸缩小为 14×14 。
- **第 2 卷积层（Conv2）：** 输入通道数为 6，输出通道数为 16，卷积核大小为 5×5 ，无填充。输出特征图尺寸为 10×10 。

- 第 2 池化层 (Pool2): 继续使用 2×2 的最大池化, 输出特征图尺寸为 5×5 。
- 展平层 (Flatten): 将 $16 \times 5 \times 5 = 400$ 个特征展平成一维向量。
- 全连接层 1 (FC1): 输入维度为 400, 输出为 120, 使用 ReLU 激活。
- 全连接层 2 (FC2): 输入维度为 120, 输出为 84, 使用 ReLU 激活。
- 输出层 (FC3): 输入为 84, 输出为 10 (对应数字 0-9 的分类)。

整个网络结构可表示为:

Input $\rightarrow$ Conv1 $\rightarrow$ ReLU $\rightarrow$ Pool1 $\rightarrow$ Conv2 $\rightarrow$ ReLU $\rightarrow$ Pool2
 $\rightarrow$ Flatten $\rightarrow$ FC1 $\rightarrow$ ReLU $\rightarrow$ FC2 $\rightarrow$ ReLU $\rightarrow$ FC3

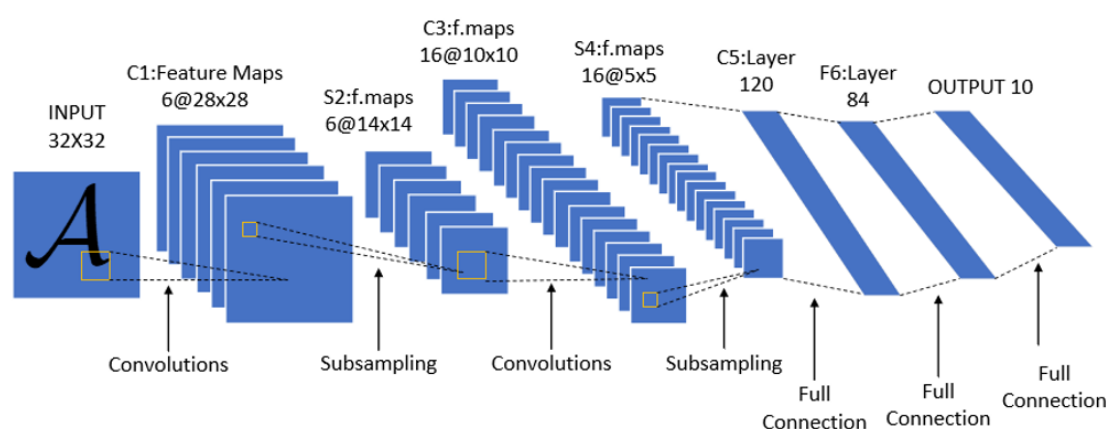


Figure 8: LeNet-5 网络结构

2.2.3 算法设计与实现

我们在不依赖高阶封装的前提下, 手动实现了 CNN 的核心组件, 搭建 Lenet 网络。

卷积层 Conv2D 使用 unfold 展开窗口实现局部区域卷积计算, 等价于标准卷积操作:

```

1 class Conv2D(torch.nn.Module):
2     def __init__(self, in_channels, out_channels, kernel_size, stride=1, padding=0):
3         super().__init__()
4         self.stride = stride # 卷积步长
5         self.padding = padding # 边缘填充
6         self.kernel_size = kernel_size if isinstance(kernel_size, tuple) else
7             ↪ (kernel_size, kernel_size)
8
9         kh, kw = self.kernel_size # 卷积核高和宽
10
11         # 使用 He 初始化方法初始化卷积核参数
12         self.weight = torch.nn.Parameter(
13             torch.randn(out_channels, in_channels, kh, kw) * (2. / (in_channels * kh *
14                 ↪ kw))**0.5

```

```

12         )
13         self.bias = torch.nn.Parameter(torch.zeros(out_channels)) # 初始化偏置为 0
14
15     def forward(self, x):
16         B, C, H, W = x.shape # 输入尺寸: 批次, 通道, 高, 宽
17         kh, kw = self.kernel_size
18
19         # 在四个方向做对称 padding
20         x_padded = torch.nn.functional.pad(x, (self.padding, self.padding, self.padding,
21         ↪ self.padding))
22
23         # 展开为局部 patch 形式 (unfold 实现滑动)
24         patches = x_padded.unfold(2, kh, self.stride).unfold(3, kw, self.stride)
25         out_h, out_w = patches.shape[2], patches.shape[3]
26
27         # 重排张量形状: 合并 patch 维度, 准备矩阵乘法
28         patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(B, out_h * out_w, -1)
29         weight_flat = self.weight.view(self.weight.shape[0], -1)
30
31         # 执行矩阵乘法加偏置
32         out = patches @ weight_flat.T + self.bias
33
34         # 重塑输出形状为卷积输出格式
35         out = out.permute(0, 2, 1).reshape(B, self.weight.shape[0], out_h, out_w)
36         return out

```

ReLU 激活函数 实现简单的 ReLU 非线性激活:

```

1 class ReLU(torch.nn.Module):
2     def forward(self, x):
3         return torch.maximum(x, torch.tensor(0.0, device=x.device)) # 大于 0 保留, 否则设
4         ↪ 为 0

```

最大池化层 MaxPool2D 基于 unfold 操作手动实现最大池化:

```

1 class MaxPool2D(torch.nn.Module):
2     def __init__(self, kernel_size, stride=None):
3         super().__init__()
4         self.kernel_size = kernel_size # 池化窗口大小
5         self.stride = stride or kernel_size # 默认步长等于窗口大小
6
7     def forward(self, x):
8         B, C, H, W = x.shape

```

```

9         k = self.kernel_size
10        s = self.stride
11
12        # 提取局部块
13        x_unf = x.unfold(2, k, s).unfold(3, k, s)
14        out_h, out_w = x_unf.shape[2], x_unf.shape[3]
15
16        # 重整形状, 最后一维为池化区域
17        x_unf = x_unf.contiguous().view(B, C, out_h, out_w, -1)
18
19        # 对池化区域取最大值
20        out, _ = x_unf.max(dim=-1)
21        return out

```

展平层 Flatten 将卷积输出拉平成一维向量, 供全连接层使用:

```

1 class Flatten(torch.nn.Module):
2     def forward(self, x):
3         return x.view(x.size(0), -1) # 将除第 0 维外全部展平

```

全连接层 Dense 手动实现线性变换层, 含参数初始化与加偏置:

```

1 class Dense(torch.nn.Module):
2     def __init__(self, in_features, out_features):
3         super().__init__()
4         self.weight = torch.nn.Parameter(
5             torch.randn(in_features, out_features) * (2. / in_features)**0.5 # He 初始化
6         )
7         self.bias = torch.nn.Parameter(torch.zeros(out_features)) # 偏置初始化为 0
8
9     def forward(self, x):
10        return x @ self.weight + self.bias # 仿照线性层公式  $y = Wx + b$ 

```

网络结构 LeNet 完整的网络结构组合如下所示, 体现标准 LeNet 架构流程:

```

1 class LeNet(torch.nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.conv1 = Conv2D(1, 6, 5, padding=2) # 第 1 卷积层: 输出通道 6
5         self.relu1 = ReLU()
6         self.pool1 = MaxPool2D(2)
7
8         self.conv2 = Conv2D(6, 16, 5) # 第 2 卷积层: 输出通道 16

```

```

9         self.relu2 = ReLU()
10        self.pool2 = MaxPool2D(2)
11
12        self.flatten = Flatten()
13        self.fc1 = Dense(16 * 5 * 5, 120) # 第 1 全连接层
14        self.relu3 = ReLU()
15        self.fc2 = Dense(120, 84) # 第 2 全连接层
16        self.relu4 = ReLU()
17        self.fc3 = Dense(84, 10) # 输出层: 10 类
18
19    def forward(self, x):
20        x = self.conv1(x); x = self.relu1(x); x = self.pool1(x)
21        x = self.conv2(x); x = self.relu2(x); x = self.pool2(x)
22        x = self.flatten(x)
23        x = self.fc1(x); x = self.relu3(x)
24        x = self.fc2(x); x = self.relu4(x)
25        x = self.fc3(x)
26        return x

```

模型训练与验证流程 本实验设计了两种不同的训练策略，以探索学习率调度对模型性能的影响。两种训练方法均基于 MNIST 手写数字分类任务，使用相同的模型结构与数据加载方式。

- 方法一（单阶段训练）：设置固定学习率 $lr=0.001$ ，连续训练 20 个 epoch。
- 方法二（分阶段调参训练）：
 - 初始阶段设置 $lr=0.005$ ，训练 10 个 epoch；
 - 然后加载保存的模型参数，使用较小学习率 $lr=0.001$ 再训练 10 个 epoch。

两种训练过程均使用交叉熵损失函数与 Adam 优化器。代码如下：

```

1  # 初始化
2  import torch
3  from torchvision import datasets, transforms
4  from torch.utils.data import DataLoader
5  from lenet_torch import LeNet
6
7  # 定义数据预处理：转张量 + 归一化
8  transform = transforms.ToTensor()
9
10 # 加载训练集和测试集
11 train_set = datasets.MNIST(root='./mnist_data', train=True, download=True,
    ↪ transform=transform)
12 test_set = datasets.MNIST(root='./mnist_data', train=False, download=True,
    ↪ transform=transform)

```

```

13
14 train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
15 test_loader = DataLoader(test_set, batch_size=64)
16
17 # 初始化模型与训练设备
18 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
19 model = LeNet().to(device)
20
21 # 第一种方式：固定 lr=0.001，训练 20 轮
22 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
23 for epoch in range(1, 21):
24     train_one_epoch(model, optimizer)
25     evaluate(model)
26
27 # 第二种方式：先用 lr=0.005 训练 10 轮
28 model = LeNet().to(device)
29 optimizer = torch.optim.Adam(model.parameters(), lr=0.005)
30 for epoch in range(1, 11):
31     train_one_epoch(model, optimizer)
32     evaluate(model)
33
34 # 加载前阶段训练参数，再以 lr=0.001 训练 10 轮
35 torch.save(model.state_dict(), "stage1.pth")
36 model.load_state_dict(torch.load("stage1.pth"))
37 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
38 for epoch in range(11, 21):
39     train_one_epoch(model, optimizer)
40     evaluate(model)

```

`train_one_epoch` 和 `evaluate` 函数分别负责模型的一轮训练与测试集评估，内部实现包括前向传播、梯度更新、准确率统计等操作。

通过对比可以发现，两种方法在初期训练阶段均能有效提高模型性能，但固定学习率的训练过程较早陷入局部收敛，准确率提升受限。而分阶段调参策略通过先用较大学习率粗调，再用较小学习率精调，有效提升了模型的最终泛化性能，且在训练后期表现更为平稳。

这种先快后稳的训练方式具有如下优点：

- 避免过早陷入局部最优；
- 训练更快收敛，且更易获得全局最优解；
- 后期微调更细致，有助于提升测试准确率和泛化能力。

说明合理设置学习率对训练效果有显著提升。

2.2.4 运行结果与分析

在本实验中，我们比较了两种不同的训练策略对模型性能的影响：固定学习率训练（方法一）与分阶段调整学习率训练（方法二）。图 9 展示了三段训练过程中的损失下降与测试准确率变化。

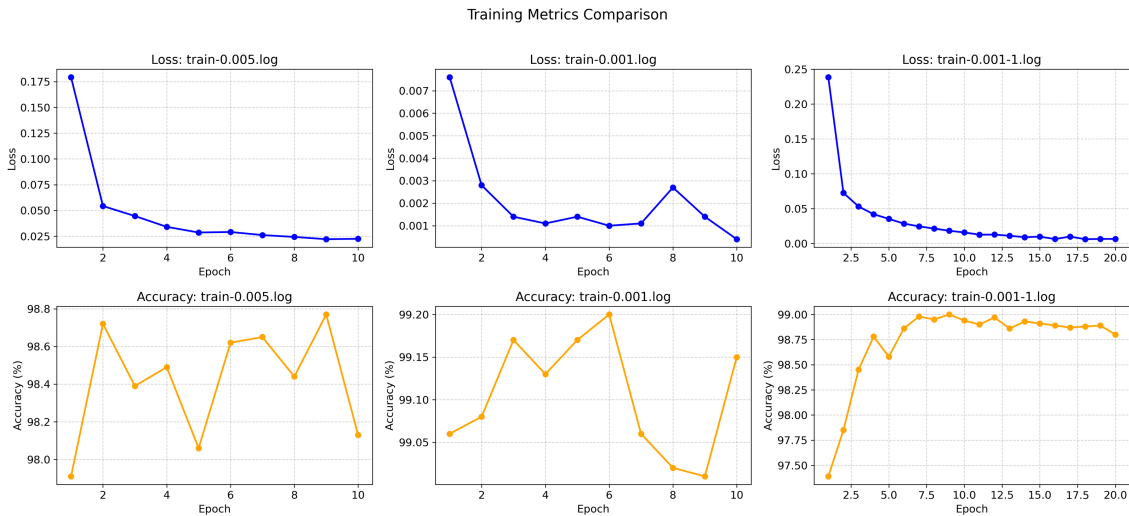


Figure 9: 三种训练阶段的损失与准确率对比：左上/左下为方法二第 1 阶段（ $lr=0.005$ ），中上/中下为方法二第 2 阶段（ $lr=0.001$ ），右上/右下为方法一（ $lr=0.001$ ）

方法一：固定学习率训练（ $lr = 0.001$ ） 该方法将学习率设为常数，连续训练 20 个 epoch。可以看到：

- 模型在第 9 个 epoch 达到 99.00% 的准确率；
- 后续准确率波动较小，趋于收敛；
- 最终稳定在 98.8% 附近，表现可靠但略有瓶颈。

方法二：分阶段调参训练（ $lr = 0.005 \rightarrow 0.001$ ） 该方法前 10 个 epoch 采用较大学习率（0.005），后 10 个 epoch 降为 0.001，策略更灵活。整体表现更优：

- 第 1 阶段迅速降低训练损失，尽管准确率存在波动（98.0%-98.8%）；
- 第 2 阶段准确率迅速攀升，在第 3 个 epoch（即总第 13 个 epoch）达到 99.17%，最终稳定在 99.20%；
- 在精度与收敛速度上均优于固定学习率方法。

通过对比可以发现，两种方法在初期训练阶段均能有效提高模型性能，但固定学习率的训练过程较早陷入局部收敛，准确率提升受限。而分阶段调参策略通过先用较大学习率粗调，再用较小学习率精调，有效提升了模型的最终泛化性能，且在训练后期表现更为平稳。

这种先快后稳的训练方式具有如下优点：

- 避免过早陷入局部最优；
- 训练更快收敛，且更易获得全局最优解；

- 后期微调更细致，有助于提升测试准确率和泛化能力。

说明合理设置学习率对训练效果有显著提升。

除了准确率外，我们还重点分析了模型训练完成后的卷积核与中间特征响应，揭示神经网络在不同层次所学习到的图像特征。

卷积核分析（Conv1） 第一层卷积核通常直接作用于原始输入图像，因此其提取的特征具有较强的可解释性。下图展示了训练完成后的 `conv1` 层的 6 个卷积核：

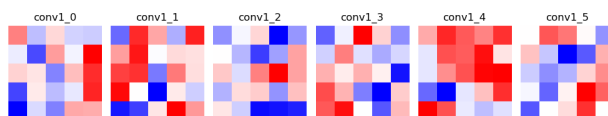


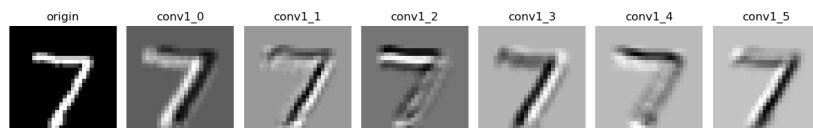
Figure 10: 第一层卷积核可视化（训练后）

对这 6 个卷积核的观察与分析如下：

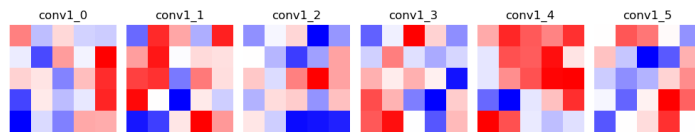
- **conv1_0**: 左侧为负，右侧为正，构成典型的“左边缘检测”核，能够响应图像中从暗到亮的水平过渡。
- **conv1_1**: 左上为正，右下为负，呈现从左上到右下的斜向梯度结构，适合检测右下方向的边缘或斜线结构。
- **conv1_2**: 中央区域权重为正，外围为弱负或接近零，构成“中心增强”核，突出图像中部区域，适合提取局部目标或高亮中心。
- **conv1_3**: 左下角为深红（强正），右上为蓝（负），方向性明显，适合提取右上的边缘或折线。
- **conv1_4**: 整块区域为红色，尤其右上权重更大，表示该核对整图亮度区域有强响应，具备整体轮廓增强功能。
- **conv1_5**: 右下为深红（强正），左上区域为浅蓝（负），与 `conv1_3` 的方向相反，倾向提取左上边缘特征，具有“左上强化”效果。

这些卷积核在结构上与经典的 Sobel、Roberts 边缘算子存在相似性，但更具多样性与任务自适应性，是训练数据驱动下自动学习得出的特征提取器。

特征响应图分析 为进一步理解卷积核在实际图像处理中的作用，我们选取了一张数字“7”的 MNIST 测试图像，使用 `visualize_feature_response.py` 脚本输出其在 `conv1` 层的中间特征图响应及对应卷积核权重。结果如图 11 所示：



(a) conv1 层二维卷积特征图响应



(b) conv1 层二维卷积层卷积核权重

Figure 11: 原始图像、conv1 层的特征图响应以及对应的卷积核（上：特征图，下：卷积核）

从上图可以观察到以下几点：

- **conv1_0**: 对图像左边界敏感，响应集中于左缘，符合其“左边缘检测”结构。
- **conv1_1**: 响应图中右下区域亮度增强，与核权重的左上正右下负结构相符，用于检测右下边界。
- **conv1_2**: 图像中心区域响应最强，边缘弱化，说明该核有较强的局部聚焦能力。
- **conv1_3**: 响应图中右侧结构激活显著，能够突出斜线或斜角笔画，符合其“右上强化”结构特征。
- **conv1_4**: 整体响应较为均匀，背景抑制良好，表明该卷积核关注目标整体亮度与形状。
- **conv1_5**: 左上区域响应最强，与权重右下高正值一致，说明该核擅长提取图像左上部分的边缘变化。

这些特征图反映了不同卷积核所提取的图像局部特征，验证了卷积层的多样性：每个卷积核负责感知图像的不同结构，如边缘、角点、纹理或亮度变化等。多样化的响应在后续层中被逐步组合，从而实现低层结构到高层语义的逐级表达。

综上所述，即使不使用预定义模板，卷积神经网络也能通过数据驱动训练自动学习出多样化且有意义的图像滤波器，提升模型对复杂图形的判别能力。这正是 CNN 在计算机视觉中表现卓越的根本原因之一。

3 进阶任务

3.1 Conv1d 实现

3.1.1 任务原理

一维卷积（Conv1D）常用于处理时间序列、自然语言、信号处理等场景，其核心思想是对输入序列滑动窗口计算内积，捕捉邻近时间步或词语之间的局部关系。其数学表达为：

$$y_t^{(o)} = \sum_{c=1}^{C_{in}} \sum_{i=0}^{k-1} w_i^{(o,c)} \cdot x_{t+i}^{(c)} + b^{(o)}$$

其中 x 为输入， w 为权重核， k 为核大小， C_{in} 为输入通道数， o 为输出通道编号。

3.1.2 算法实现

1. 实现思路 我们使用 PyTorch 编写了自定义模块 Conv1D，其核心步骤：

- 使用 F.pad 添加边界填充（padding）；
- 使用 unfold 将输入序列展开为滑动窗口；
- 通过张量重排与矩阵乘法实现高效前向传播；
- 支持自定义输入通道、输出通道、卷积核大小、步长与边界填充。

2. 代码实现 以下为核心类的定义代码：

```
1 class Conv1D(nn.Module):
2     def __init__(self, in_channels, out_channels, kernel_size, stride=1, padding=0):
3         super().__init__()
4         self.weight = nn.Parameter(
5             torch.randn(out_channels, in_channels, kernel_size) * (2. / (in_channels *
6                 ↪ kernel_size))**0.5
7         )
8         self.bias = nn.Parameter(torch.zeros(out_channels))
9
10    def forward(self, x):
11        x_padded = F.pad(x, (self.padding, self.padding))
12        x_unf = x_padded.unfold(2, self.kernel_size, self.stride)
13        x_unf = x_unf.permute(0, 2, 1, 3).reshape(B, L_out, -1)
14        out = x_unf @ self.weight.view(self.weight.shape[0], -1).T + self.bias
15        return out.permute(0, 2, 1)
```

3.1.3 实验验证

为了验证自定义 Conv1D 实现的正确性，我们设计了两个实验：

1. 与官方 nn.Conv1d 实现的对比实验 使用相同权重与输入，比较自定义模块与 PyTorch 官方模块输出是否一致。测试结果为：

All close: True

表明手动实现的 Conv1D 数值等价于官方实现，计算逻辑正确。

```
1 def test_conv1d():
2     x = torch.randn(2, 3, 10) # batch=2, in_channels=3, seq_len=10
3     in_c, out_c, k, s, p = 3, 4, 3, 1, 1
4
5     # 初始化自定义卷积与官方 Conv1d
6     my_conv = Conv1D(in_c, out_c, k, stride=s, padding=p)
```

```

7     torch_conv = nn.Conv1d(in_c, out_c, k, stride=s, padding=p)
8
9     # 复制参数，确保输入一致性
10    torch_conv.weight.data = my_conv.weight.data.clone()
11    torch_conv.bias.data = my_conv.bias.data.clone()
12
13    # 分别前向传播
14    y1 = my_conv(x)
15    y2 = torch_conv(x)
16
17    # 判断数值误差是否足够小
18    print("All close:", torch.allclose(y1, y2, atol=1e-5))

```

2. 语义模式检测实验 构造一个 4 词句子：“not good service terrible”，对应情感值 $[-0.8, 0.9, 0.1, -0.9]$ ，并使用自定义卷积核 $[-1, +1]$ 来检测“负 $\rightarrow$ 正”结构（如“not good”）。

卷积输出如图 12 所示，在“not good”位置给出了最大正响应。

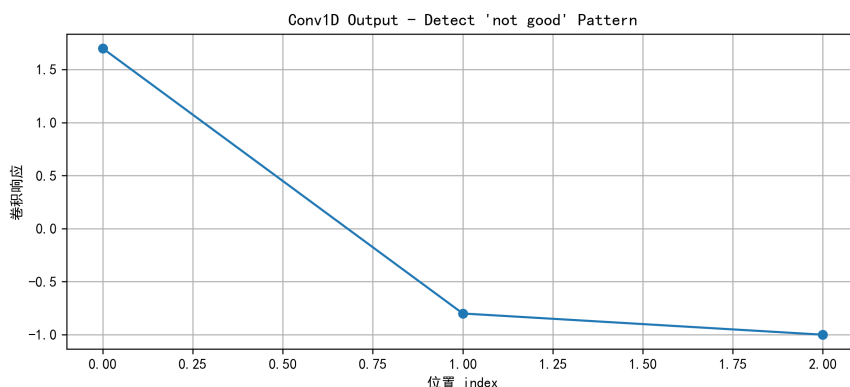


Figure 12: 自定义 Conv1D 检测“not good”模式输出

3.1.4 总结

通过上述两个验证实验，我们确认手动实现的一维卷积层在数值准确性与应用功能上均满足预期。结合文本卷积情感分析任务，Conv1D 层可用于构建轻量级自然语言分类模型、事件检测器等，具有广泛的应用前景。

3.2 Conv3d 实现

3.2.1 任务目标

三维卷积在视频分析（时空维度）、医学影像处理（3D 空间）等场景具有重要应用价值。本任务在二维卷积（Conv2d）实现的基础上拓展至三维卷积，要求实现支持多通道输入输出、可配置三维核尺寸（kernel size）、三维步长（stride）、三维填充（padding）的卷积核，能够处理五维张量 (N, C, D, H, W) 形式的数据，并通过与 PyTorch 官方 Conv3d 进行对比验证正确性。

3.2.2 原理分析

三维卷积的核心思想是将卷积核的滑动窗口从二维平面扩展到三维空间，使其能够在深度（D）、高度（H）、宽度（W）三个维度上同时提取特征。相较于二维卷积的平面特征提取，三维卷积在深度维度（D）增加滑动感知能力，进而实现对体积数据的特征建模。

具体而言，在视频处理中，深度维度（D）对应连续帧的时间序列，从而使三维卷积核能够同时捕获空间与时间上的动态特征，如动作特征捕捉；在医学领域 CT 影像中，深度维度（D）则可以表示人体横断面的分层扫描，从而使三维卷积核能够同时捕获三维空间的结构特征，例如血管在连续切片中的走向模式。

参考二维卷积，拓展一个维度即可得到三维卷积的数学表达如下：

$$O_{c',d_o,h_o,w_o} = \sum_{c=0}^{C_{in}-1} \sum_{i=0}^{k_d-1} \sum_{j=0}^{k_h-1} \sum_{k=0}^{k_w-1} W_{c',c,i,j,k} \cdot X_{c,s_d d_o+i,s_h h_o+j,s_w w_o+k} + b_{c'}$$

其中， O_{c',d_o,h_o,w_o} 表示输出张量在第 c' 个输出通道、深度索引 d_o 、高度索引 h_o 、宽度索引 w_o 处的值； $W_{c',c,i,j,k}$ 为第 c' 个输出通道与第 c 个输入通道之间的三维卷积核权重， i, j, k 分别对应深度、高度、宽度方向的核内索引； $X_{c,s_d d_o+i,s_h h_o+j,s_w w_o+k}$ 表示输入张量在第 c 个通道、深度 $s_d d_o + i$ 、高度 $s_h h_o + j$ 、宽度 $s_w w_o + k$ 处的值，其中 s_d, s_h, s_w 分别为深度、高度、宽度方向的步长； k_d, k_h, k_w 分别为卷积核在深度、高度、宽度方向的尺寸； $b_{c'}$ 为第 c' 个输出通道的偏置项。

3.2.3 算法实现

延续 Conv2d 二维卷积中，先向量化展开，后再进行矩阵乘法的实现方式。主要变化在于从二维滑动展开变为三维滑动窗口展开。核心实现代码（前向传播）如下：

```
1 def _conv3d_manual_einsum(self, x, weight, bias, stride, padding):
2     N, Cin, D_in, H_in, W_in = x.shape
3     Cout, _, kd, kh, kw = weight.shape
4
5     # 计算输出三维尺寸
6     D_out = (D_in + 2*padding[0] - kd) // stride[0] + 1
7     H_out = (H_in + 2*padding[1] - kh) // stride[1] + 1
8     W_out = (W_in + 2*padding[2] - kw) // stride[2] + 1
9
10    # 三维 padding 处理
11    x_padded = F.pad(x, (padding[2], padding[2], padding[1], padding[1],
12                          padding[0], padding[0]), mode=self.mode)
13
14    # 三维 unfold 展开为六维张量
15    windows = x_padded.unfold(2, kd, stride[0]) # 深度维度展开
16                      .unfold(3, kh, stride[1]) # 高度维度展开
17                      .unfold(4, kw, stride[2]) # 宽度维度展开
18    windows = windows.contiguous().view(N, Cin, D_out, H_out, W_out, -1)
19
20    # 爱因斯坦求和实现批量卷积
```

```

21     weight_flat = weight.view(Cout, Cin, kd*kh*kw)
22     output = torch.einsum('ncdhwk,ock->nodhw', windows, weight_flat)
23
24     # 偏置项广播加法
25     if bias is not None:
26         output += bias.view(1, -1, 1, 1, 1)
27     return output

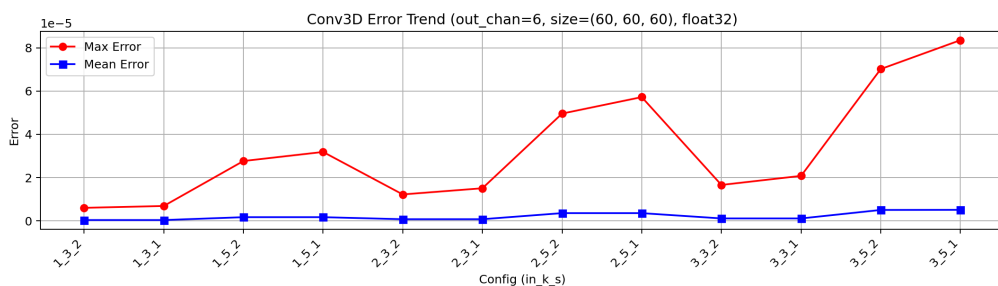
```

代码流程说明:

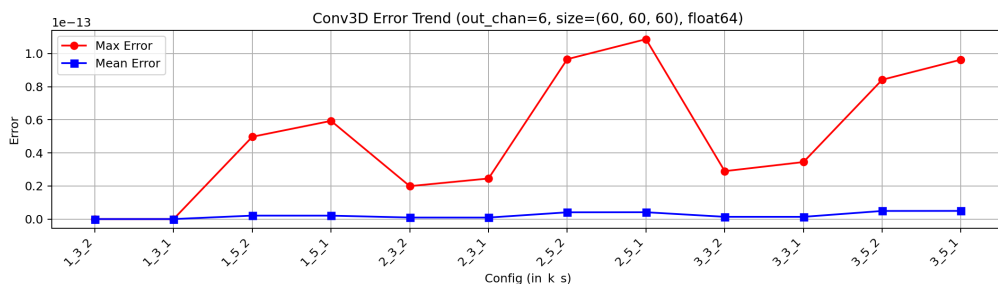
1. 函数中先计算输出尺寸 (Dout,Hout,Wout)，再根据 padding 模式对输入张量进行三维填充；
2. 分别对深度维度、高度维度、宽度维度进行滑动窗口提取，`unfold(2, kd, stride[0])`，`unfold(3, kh, stride[1])`，`unfold(4, kw, stride[2])`，从而展开为六维张量，尺寸为 (N,Cin,D',H',W',kd*kh*kw)；
3. 张量先按尺寸 (Cout, Cin, kd*kh*kw) 重塑改变形状，而后使用爱因斯坦求和记号，将滑动窗口与卷积核权重矩阵对应相乘，最后再添加偏置量；
4. 输出形状为 (N,Cout,Dout,Hout,Wout)，即为所需的三维卷积结果。

3.2.4 正确性验证

为验证实现的正确性，我们对不同输入通道数 ($\text{in_channels} = 1, 2, 3$)、卷积核尺寸 ($k = 3, 5$)、步长 ($s = 1, 2$) 的组合情况进行测试，输入尺寸固定为 $60 \times 60 \times 60$ ，输出通道数固定为 6。每组参数配置下，随机生成 10 对输入张量和卷积核，再用手动设计的 Conv_3d 与 PyTorch 官方 Conv3d 分别对每对输入张量和卷积核进行卷积计算，统计每组中两者运算结果的最大误差和平均误差，并绘制各配置下的误差均值与最大值趋势图如下。



(a) 单精度浮点误差分布



(b) 双精度浮点误差分布

Figure 13: Conv_3d 的手动实现与官方库的数值误差对比

实验表明，单精度下最大绝对误差 $8.35e-5$ ，平均误差 $5.08e-6$ ，误差量级与硬件浮点舍入误差相当；双精度下最大绝对误差降至 $1.09e-13$ 以下，验证了算法实现的数学正确性。

3.2.5 自定义卷积核效果

1. 卷积核设计与实现 基于视频处理的时空特性，我们设计了六类三维卷积核，其数学形式与功能如下：

Table 1: 基于 Kronecker 积的卷积核表示

卷积核名称	功能描述	数学表达
temporal_avg_2d	平面平均平滑，用于基础降噪	$\mathcal{K} = \frac{1}{9} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$
gaussian_2d	平面高斯平滑，保留边缘的降噪	$\mathcal{K} = \frac{1}{16} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$
sharpen_2d	平面锐化，增强图像细节	$\mathcal{K} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$
sharpen_3d	时空锐化，提升视频清晰度	$\mathcal{K} = \frac{1}{3} \begin{bmatrix} -1 \\ 5 \\ -1 \end{bmatrix} \otimes \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$
motion_grad	时间差分检测运动方向	$\mathcal{K} = \begin{bmatrix} +1 \\ 0 \\ -1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$
edge_3d	时空边缘检测，捕捉运动轮廓	$\mathcal{K} = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \otimes \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$

符号说明：

- $\otimes$: Kronecker 积, $(\mathbf{t} \otimes \mathbf{S})_{d,h,w} = t_d \cdot S_{h,w}$
- $\mathbf{t} \in \mathbb{R}^3$: 时间/深度通道维度向量
- $\mathbf{S} \in \mathbb{R}^{3 \times 3}$: 平面空间卷积核矩阵
- $\mathcal{K} \in \mathbb{R}^{3 \times 3 \times 3}$: 三维卷积核张量

卷积核定义代码

```

1 kernels = {
2     "temporal_avg_2d": torch.stack([
3         torch.zeros((3,3)),
4         torch.ones((3,3))/9,
5         torch.zeros((3,3))
6     ]),

```

```

7     "gaussian_2d": torch.stack([
8         torch.zeros((3,3)),
9         torch.tensor([[1,2,1],[2,4,2],[1,2,1]])/16,
10        torch.zeros((3,3))
11    ]),
12    "sharpen_2d": torch.stack([
13        torch.zeros((3,3)),
14        torch.tensor([[0,-1,0],[-1,5,-1],[0,-1,0]]),
15        torch.zeros((3,3))
16    ]),
17    "sharpen_3d": torch.stack([
18        torch.tensor([[0,-1,0],[-1,5,-1],[0,-1,0]])*-1,
19        torch.tensor([[0,-1,0],[-1,5,-1],[0,-1,0]])*5,
20        torch.tensor([[0,-1,0],[-1,5,-1],[0,-1,0]])*-1
21    ])/3,
22    "motion_grad": torch.stack([
23        torch.full((3,3),1),
24        torch.zeros((3,3)),
25        torch.full((3,3),-1)
26    ]),
27    "edge_3d": torch.einsum('hw,t->thw',
28        torch.tensor([[[-1,0,1],[-2,0,2],[-1,0,1]]],
29        torch.tensor([1,0,-1])
30    )
31 }

```

2. 量化处理模块

在三维卷积处理后引入量化步骤，目的是为后续锐化操作提供更平滑的输入。通过量化可有效抑制微小噪声，突出主要结构，使锐化结果更加自然、细腻，避免噪声在多次操作中被过度放大。

量化的原理是将连续像素或特征值按设定步长分段映射到有限的离散等级，实现色彩空间压缩。步长越大，离散化程度越高，细节越被平滑化。例如，步长为 8 时，像素值将被分为 32 个等级，图像呈现略明显的分色效果，有助于提升后续处理效果。

量化的意义在于：

- 降噪与突出特征：抑制微小变化，突出主要结构和运动边缘
- 数据压缩：减少存储带宽和后续处理压力
- 风格化效果：高量化步长可产生卡通化、分色等视觉效果

代码实现如下：

```

1 def applyKernels(inputTensor: torch.Tensor, Operations: list[tuple]) -> torch.Tensor:
2     output = inputTensor

```



```

3     for name, quantize in Operations:
4         kernel = kernels[name] # 从工厂获取卷积核
5         output = conv3d(output, kernel) # 应用三维卷积
6
7         # 量化处理：压缩色彩空间
8         if quantize:
9             output = ((output * 256).round() // quantize * quantize) / 256
10    return output

```

- **量化参数 quantize:** 控制色彩精度（如 16→16 色阶，8→32 色阶），量化步长越大，色彩层次越少，降噪和风格化效果越明显
- **操作链 Operations:** 支持任意顺序的卷积核组合与量化操作，灵活构建多步处理流程

3. 处理方案 为展示三维卷积核处理的效果，我们基于视频处理逻辑，对两幅动图分别进行了针对降噪和运动检测两类任务的处理，设计 8 种处理流程如下：

Table 2: 动图处理方案配置

任务类型	输出名称	处理流程
降噪处理	空间平均降噪	1 次 16 量化平均降噪 + 5 次无量化平均降噪
	空间锐化	空间平均降噪 → 1 次平面锐化
	空间高斯降噪	1 次 8 量化高斯降噪 + 5 次无量化高斯降噪
	空间锐化	空间高斯降噪 → 1 次平面锐化
运动检测	运动梯度	1 次 8 量化运动梯度检测
	降噪 + 时空锐化	运动梯度 → 3 次 8 量化平均降噪 + 时空锐化
	运动边缘	1 次 8 量化时空边缘检测
	降噪 + 时空锐化	时空边缘 → 3 次 8 量化平均降噪 + 时空锐化

具体操作链集合如下：

```

1 # 三个色彩通道用一样的卷积核
2 # 降噪.gif: 1. 先使用空间平均核进行降噪，再使用空间锐化核提升细节
3 # 降噪.gif: 2. 先使用空间高斯核进行降噪，再使用空间锐化核提升细节
4 # 运动.gif: 1. 先使用运动梯度核检测运动方向，再使用时空锐化核提升细节
5 # 运动.gif: 2. 先使用运动边缘检测，再使用时空锐化核提升细节
6 Tasks = [
7     {
8         "src": " 降噪.gif",
9         "op": [(["temporal_avg_2d",16)] * 1 + [(["temporal_avg_2d",0)] * 5,

```

```

10     "dst": " 降噪 1m.gif",
11 },
12 {
13     "src": " 降噪.gif",
14     "op": [ ("temporal_avg_2d",16)] * 1 + [ ("temporal_avg_2d",0)] * 5 \
15           + [ ("sharpen_2d",0)] * 1,
16     "dst": " 降噪 1f.gif",
17 },
18 {
19     "src": " 降噪.gif",
20     "op": [ ("gaussian_2d",8)] * 1 + [ ("gaussian_2d",0)] * 5,
21     "dst": " 降噪 2m.gif",
22 },
23 {
24     "src": " 降噪.gif",
25     "op": [ ("gaussian_2d",8)] * 1 + [ ("gaussian_2d",0)] * 5 \
26           + [ ("sharpen_2d",0)] * 1,
27     "dst": " 降噪 2f.gif",
28 },
29 {
30     "src": " 运动.gif",
31     "op": [ ("motion_grad",8)],
32     "dst": " 运动 1m.gif",
33 },
34 {
35     "src": " 运动.gif",
36     "op": [ ("motion_grad",8)] + [ ("temporal_avg_2d",8)] * 3 \
37           + [ ("sharpen_3d",0)],
38     "dst": " 运动 1f.gif",
39 },
40 {
41     "src": " 运动.gif",
42     "op": [ ("edge_3d",8)],
43     "dst": " 运动 2m.gif",
44 },
45 {
46     "src": " 运动.gif",
47     "op": [ ("edge_3d",8)] + [ ("temporal_avg_2d",8)] * 3 + [ ("sharpen_3d",0)],
48     "dst": " 运动 2f.gif",
49 }
50 ]

```

4. 效果可视化及对比 处理结果如下图 14 和图 15 所示:



Figure 14: 降噪处理效果对比: (左) 原动图 (中) 空间平均降噪/空间高斯降噪 (右) 降噪 + 锐化



Figure 15: 运动检测效果对比: (左) 原动图 (中) 运动梯度/运动边缘 (右) 降噪 + 时空锐化

布局说明如下:

- 第一列: 原始输入动图帧截取
- 第二列 (上): 对应帧基础降噪/运动检测结果 (1m)
- 第二列 (下): 对应帧优化降噪/边缘检测结果 (2m)
- 第三列 (上): 对应帧基础降噪/运动检测 + 锐化结果 (1f)
- 第三列 (下): 对应帧优化降噪/边缘检测 + 锐化结果 (2f)

整体效果: 我们设计的各类三维卷积核均能有效实现其预期功能: 空间降噪核能平滑噪声、锐化核能增强细节、运动检测核能突出时序变化, 实验结果表明这些自定义核在实际图像与视频处理中表现出良好的特征提取能力, 佐证了手动实现的三维卷积函数的正确性。

效果对比：

1. **降噪效果优化：**高斯降噪（2m）较平均降噪（1m）显著提升细节保留能力，在消除噪声的同时减少特征损失。
2. **纹理恢复能力：**锐化处理（*f）有效恢复降噪过程中损失的纹理特征，使边缘结构更鲜明（对比 1f/2f 与 1m/2m）。
3. **运动检测优势：**时空边缘检测（运动 2m）比运动梯度（运动 1m）更清晰地捕捉物体运动轮廓。
4. **量化参数提升平滑度：**经过实验验证与参数调整，可以发现我们选定的量化参数可以有效提升图像平滑度，减少色彩断层或反常噪点现象。

3.3 性能测试与分析

3.3.1 测试目标与方法

1. **测试目标：**验证手动实现的卷积操作在数值精度与计算效率上是否满足要求，并与 PyTorch 官方实现进行全面对比分析，重点考察：

- 不同精度（float32/float64）下的数值稳定性
- 不同数据规模和配置下的计算性能
- 手动实现方案的实际优化效果

2. **测试方法：**

- **测试环境配置**
 - 硬件：NVIDIA GeForce RTX 4080 Laptop GPU (12GB GDDR6X)
 - 软件：PyTorch 2.7.0 + CUDA 12.8（启用 CuDNN 确定性计算和基准优化）
 - 随机种子：固定随机种子 (`torch.manual_seed(42)`) 保证结果可复现
- **测试数据集**
 - 正态分布随机张量（`torch.randn` 生成）
 - 单精度/双精度分别测试（float32 与 float64）
- **参数配置方案**
 - 正确性测试：如下表 3 所示，覆盖多种配置组合
 - 性能测试：如下表 4 所示，固定主要参数，变化规模
- **度量指标与方法**
 - 数值精度：输出张量绝对误差统计（最大/平均/最小值）：

$$\text{误差} = |\text{手动输出} - \text{PyTorch 输出}|$$

- 计算性能：计时覆盖三个方面：
 - * 前向传播时间
 - * 反向传播时间
 - * 完整步骤时间（前向 + 反向）
- 计时方法：启用 `torch.cuda.synchronize()` 确保精确性
- 结果统计：重复 10 次实验取均值消除波动

Table 3: 正确性测试参数配置

参数	Conv1D	Conv3D
输入尺寸	200,000	60×60×60
输入通道	{1,2,3}	
卷积核尺寸	{3,5}	
步长 (stride)	{1,2}	
输出通道数	6	
测试次数	10 组取误差统计	

Table 4: 性能测试参数配置

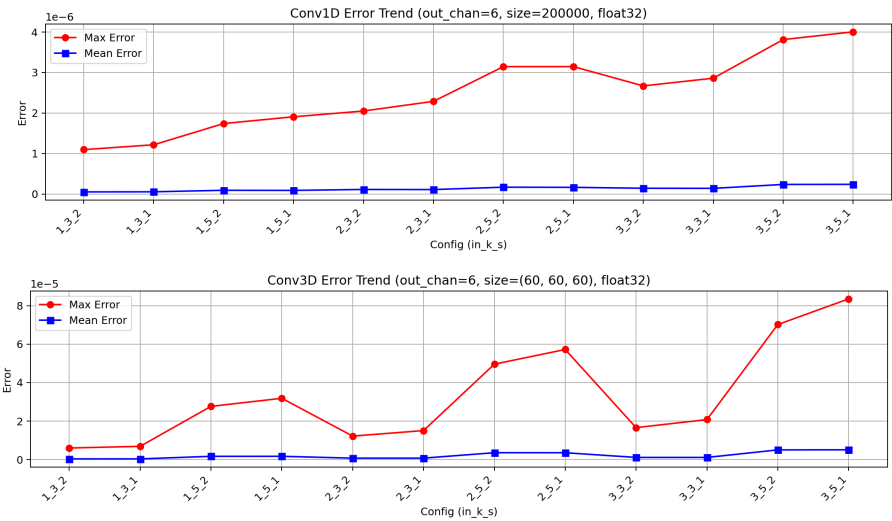
参数	Conv1D	Conv3D
输入尺寸	1,000,000	64×64×64
	5,000,000	128×128×128
	10,000,000	192×192×192
	20,000,000	
运行次数	10 次取平均	
输入通道数	3	
输出通道数	12	
卷积核尺寸	3	3×3×3
步长 (stride)	1	
填充 (padding)	1	

3.3.2 正确性测试结果

Table 5: 卷积操作精度测试结果（最差配置）

类型	精度	最差配置	误差统计
Conv1D	float32	(in=3, k=5, s=1)	max: 4.01×10^{-6}
			mean: 2.39×10^{-7}
	float64	(in=3, k=5, s=1)	max: 3.46×10^{-15}
			mean: 2.51×10^{-19}
Conv3D	float32	(in=3, k=5, s=1)	max: 8.35×10^{-5}
			mean: 5.08×10^{-6}
	float64	(in=2, k=5, s=1)	max: 1.09×10^{-13}
			mean: 4.13×10^{-15}

Figure 16: 不同卷积类型在 float32 下的误差分布趋势



正确性测试结果表明：

- **整体误差极小，数值可靠：**在默认单精度下，所有手动实现的卷积操作与 PyTorch 官方实现卷积操作运算结果的最大/平均误差均较低，可视为系统误差，充分证明了算法的正确性和工程可用性。
- **误差与数据精度密切相关：**精度从单精度（float32）提升到双精度（float64）时，所有误差可进一步降低至 10^{-13} 量级以下，满足高精度科学计算需求。
- **卷积核尺寸增大时，误差略有上升：**随着核尺寸变大，累计浮点误差会显著增加，但整体仍远低于实际应用的容忍阈值。

- 步长增大时，误差略有下降：步长增加导致输出点数减少，累计误差随之降低，数值稳定性提高。
- **Conv1D 与 Conv3D 误差差异**：Conv1D 和 Conv3D 测试输入数据量相当，但 Conv1D 的误差整体比 Conv3D 小一个数量级以上。主要原因在于三维卷积每个输出点涉及的累加项远多于一维卷积，浮点加法的舍入误差在高维度下更易累积放大，因此 Conv3D 整体误差略高，但二者均远低于实际应用错误阈值，数值精度依然可靠。

3.3.3 性能对比及分析

1. 性能对比

Figure 17: 手动实现与 PyTorch 官方实现的性能对比（单精度）

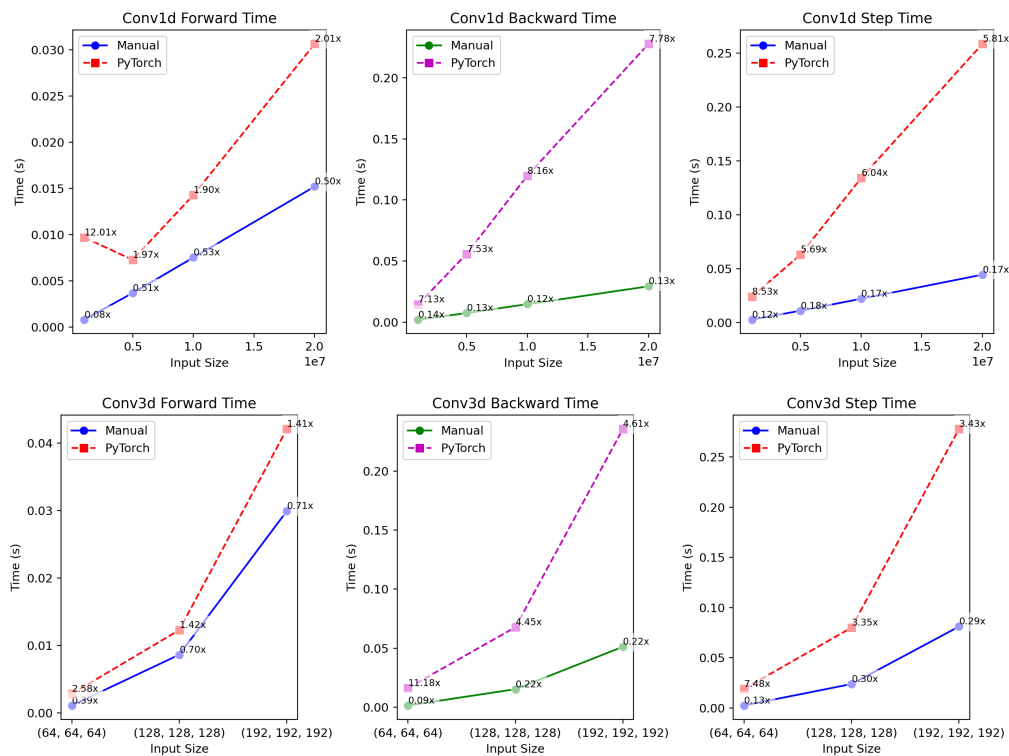


Table 6: 最大规模下计算性能对比

类型	数据规模	手动实现 (ms)	PyTorch(ms)	加速比
Conv1D	20,000,000 点	44.5	258.5	5.81×
Conv3D	192 × 192 × 192 体素	81.1	277.8	3.43×

关键性能特征：

- **显著加速**：最坏情况下，手动实现的 Conv1D 单步运算仍获 5.69× 加速，Conv3D 仍获 3.43× 加速。

- **加速效果稳定：**无论数据规模大小，手动实现均显著快于官方实现，且随着规模增大，加速比趋于正向稳定。
- **非对称优化：**无论 Conv1D 还是 Conv3D，手动实现的反向传播加速比均显著高于前向传播。例如，最大规模下 Conv1D 前向/反向加速比分别为 $2.01\times/7.78\times$ ，Conv3D 前向/反向加速比分别为 $1.41\times/4.61\times$ ，反向传播的优化效果更突出。

2. 性能特征及优化原理分析

- **内存访问优化：**手动实现通过 `unfold` 操作将输入数据转换为矩阵 (`im2col`)，然后使用矩阵乘法 (GEMM) 完成卷积计算。这种方法虽然增加了内存占用，但能将卷积操作转化为高度优化的矩阵乘法，从而充分利用现代 CPU/GPU 的缓存和并行计算能力。PyTorch 的官方实现在某些情况下（尤其是小规模或非标准配置）可能选择了较慢的算法，如直接卷积或 FFT 卷积，而我们的实现始终使用 GEMM。
- **减少内存分配和复制：**手动实现中，我们通过 `unfold` 和 `view` 操作，其中后者可以避免不必要的中间张量，并尽量利用原位操作。此外，我们将权重重塑为矩阵，直接使用矩阵乘法，使得整个计算过程更加线性。而 PyTorch 的卷积操作可能需要为不同算法分配临时缓冲区，增加了内存分配开销。
- **高效的矩阵乘法：**手动实现使用 `torch.einsum`（在 Conv3d 中）或简单的矩阵乘法（在 Conv1d 中），这些操作在 PyTorch 中由高度优化的 BLAS 库（如 MKL、OpenBLAS 或 CUDA 的 cuBLAS）支持。而 PyTorch 的卷积操作可能使用了较慢的卷积算法。
- **输入规模的影响：**随着输入规模增大，手动实现的性能优势减小并趋向稳定（Conv1D 约为 6 倍，Conv3D 约为 3.5 倍）。这是因为大规模矩阵乘法更能够发挥 BLAS 库的并行能力。同时，PyTorch 官方实现在小规模时可能使用了启动开销较大的算法（如 FFT），而在大规模时切换到 GEMM，从而导致规模增大时性能差异减小。但手动实现的 GEMM 可能比 PyTorch 的 GEMM 实现更直接和高效（没有额外的封装开销），从而保持了性能优势。
- **维度的影响：**Conv3d 的加速比低于 Conv1d，原因可能是三维卷积的 `im2col` 操作会产生更大的矩阵（内存占用高），且矩阵乘法中矩阵的尺寸可能不是最优（例如矩阵太大导致无法充分利用缓存，或太小导致并行度不够）。在 Conv3d 中，我们使用了 `unfold` 两次（先展开 D，再展开 H，再展开 W），然后组合成一个大的矩阵，这个过程需要多次内存重排，可能带来额外开销，从而导致性能下降。而 Conv1d 的 `unfold` 操作相对简单，直接将一维数据转换为二维矩阵，内存布局更友好，这些都使得手动实现 Conv3d 较官方版本的加速比不如 Conv1d。
- **非对称优化：**我们发现前向传播的加速比低于反向传播。这可能是因为手动实现的反向传播同样利用高效 BLAS 库进行矩阵运算（如权重的梯度计算：输入转置乘输出梯度）。然而，PyTorch 的自动微分系统在反向传播时需要计算多个梯度（如权重、偏置、输入），并可能存储前向过程的中间结果以支持梯度计算，这带来了额外的计算和内存开销。相比之下，手动实现能够精确控制并仅计算所需梯度（例如仅权重梯度），消除了这些冗余操作，从而在反向传播中获得更显著的加速比。
- **其他因素：**PyTorch 的卷积操作考虑了更广泛的边界情况（如不同的 padding 模式、膨胀、分组等），这可能导致其实现中包含了很多条件判断，降低了性能。而手动实现只支持特定

的配置（如我们的实现只支持步长和 padding 为常量，不支持膨胀和分组），因此可以针对这些情况做简化，降低了内存占用和计算开销。

3.4 自定义卷积核设计

3.4.1 任务问题

在深度学习兴起前，人工设计的卷积核是图像处理的核心工具，用于提取底层视觉特征，具有可解释性强、计算高效等优势，至今仍在小数据和无训练需求的场景中发挥作用。

我们的任务是：根据卷积核的原理，设计固定 5*5 卷积核，使其分别具备**增强中心**、**波纹检测**、**突出高频细节**、**高斯模糊**功能，并将其可视化。除此之外，探索核参数与特征响应之间的映射关系，分析核结构对图像处理效果的影响机制。

3.4.2 原理分析

卷积核的本质是通过局部加权求和，提取输入数据的空间特征，其权重分布直接决定了卷积操作的特定功能。为此，了解每个功能实质对于卷积核的设计尤为重要。

- **增强中心核**：受人类视网膜中央凹启发，核心目标是通过空间权重分配强化图像中心区域的响应，形成局部对比度增强效果。其数学本质是构建一个径向衰减的权重分布函数，满足：

$$K(i, j) = f(\sqrt{(i - c)^2 + (j - c)^2}), c = \lfloor k/2 \rfloor$$

其中 f 通常选择指数衰减函数或者线性衰减函数。

设计的卷积核应从核矩阵中心向边缘呈指数递减，中心点权重最高，以增强对具有“中心对称”的图形特征进行提取。下面是一个示例卷积核：

$$\begin{bmatrix} -1 & -1 & -1 & -1 & -1 \\ -1 & 1 & 2 & 1 & -1 \\ -1 & 2 & 4 & 2 & -1 \\ -1 & 1 & 2 & 1 & -1 \\ -1 & -1 & -1 & -1 & -1 \end{bmatrix}$$

- **波纹检测核**：受人类视觉皮层简单细胞多方向整合及相位不变性启发，核心目的是检测图像中具有周期性方向纹理的特征。其数学本质是构建一个方向敏感的带通滤波器，满足：

$$K(i, j) = \sin\left(\frac{2\pi}{\lambda}(i \cos \theta + j \sin \theta)\right) \cdot e^{-\frac{d^2}{2\sigma^2}}$$

其中波长 λ 控制波纹间距，角度 θ 控制波纹方向，高斯衰减用于抑制边缘响应。

设计时应考虑目标纹理的识别方向，并且可以多方向复合来增强卷积核的性能。下面是一个多方向复合增强型的波纹检测核示例：

$$\begin{bmatrix} 1 & 0 & -1 & 0 & 1 \\ 0 & -2 & 0 & -2 & 0 \\ -1 & 0 & 6 & 0 & -1 \\ 0 & -2 & 0 & -2 & 0 \\ 1 & 0 & -1 & 0 & 1 \end{bmatrix}$$

- **高频细节突出核：**受视网膜神经节细胞中心兴奋与周边抑制的拮抗作用启发，核心目的是通过中心强化-边缘抑制机制增强图像中的高频成分。其数学本质是构建一个改进的拉普拉斯算子，满足：

$$K(i, j) = \underbrace{G\delta_{(c,c)}}_{\text{中心增益}} - \underbrace{\alpha \sum_{n \in N} w_n}_{\text{周边抑制}}$$

其中 G 为中心增益系数， α 为周边衰减因子， N 为邻域像素集合。

设计的时候应注意权重分配的物理合理性以及频率选择。下面是一个示例卷积核：

$$\begin{bmatrix} -1 & -1 & -1 & -1 & -1 \\ -1 & 0 & 0 & 0 & -1 \\ -1 & 0 & 24 & 0 & -1 \\ -1 & 0 & 0 & 0 & -1 \\ -1 & -1 & -1 & -1 & -1 \end{bmatrix}$$

- **高斯模糊核：**受人类视觉系统对高频噪声不敏感、神经节细胞感受野对边缘信息进行预处理启发，核心目的是通过高斯平滑来降低图像噪声。其数学本质是构建一个二维高斯函数，满足：

$$K(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

其中 σ 为控制模糊强度的标准差。

设计时应注意 σ 的物理含义，选取合适的参数。下面为一个卷积核的示例：

$$\frac{1}{273} \times \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$

（注：实际处理过程中对其参数进行微调，以增强模糊效果，与传统“高斯模糊核”有一定差异）

3.4.3 算法设计与实现

经过上面的原理分析，按照步骤设计算法，即可生成参数可调的卷积核。

- **增强中心核：**

主要参数如下：

- 核尺寸 *size*：确定作用区域范围。大核适合全局增强，小核适合精细调整。
- 中心增益系数 *center_gain*：控制中心点强化强度，值越大，中心越亮。
- 衰减系数 *decay*：决定权重向外衰减的系数。值越小衰减越慢，作用范围越大。

核的生成过程为：

先计算距离，再分配权重，最后归一化防止亮度偏移。

生成代码如下：

```

1     def central_enhance_kernel(size=5, center_gain=1.5, decay=0.3):
2         c = size // 2
3         kernel = np.zeros((size, size))
4         for i in range(size):
5             for j in range(size):
6                 distance = np.sqrt((i-c)**2 + (j-c)**2)
7                 kernel[i,j] = max(0, center_gain - decay*distance)
8     return kernel / kernel.sum()

```

- 波纹检测核:

主要参数如下:

- 核尺寸 *size*
- 波长 *wavelength*: 控制核匹配的波纹空间周期。当 $\lambda \approx$ 目标纹理周期时, 响应强度最大。
- 角度 *angle*: 设定核的检测方向, 应与目标纹理方向对齐。
- 衰减系数 *sigma*: 控制核的空间作用范围。值越小方向选择性越强但抗噪性减弱。
- 相位 *phi*: 控制波纹相位偏移, 0 为 sin 型核, $\pi/2$ 为 cos 型核。

核的生成过程为:

先计算各点到中心的距离, 再根据正弦函数和高斯衰减分配方向敏感权重, 最后归一化。

生成代码如下:

```

1     def generate_ripple_kernel(size=5, wavelength=6, angle=45, sigma=1.0, phi=0):
2         c = size // 2
3         kernel = np.zeros((size, size))
4         angle_rad = np.deg2rad(angle)
5         for i in range(size):
6             for j in range(size):
7                 x, y = i - c, j - c
8                 proj = x * np.cos(angle_rad) + y * np.sin(angle_rad)
9                 # 波纹函数 + 高斯衰减
10                kernel[i,j] = np.sin(2 * np.pi * proj / wavelength + phi) * \
11                    np.exp(-(x**2 + y**2) / (2 * sigma**2))
12        kernel = kernel / np.max(np.abs(kernel))
13    return kernel

```

- 高频细节突出核:

主要参数如下:

- 核尺寸 *size*
- 中心点权重 *center_gain*: 正数, 控制中心点的增强强度。值越大边缘越锐利。

- 周边权重 *surround_weight*: 负数, 控制周边区域抑制强度。绝对值越大, 背景抑制越强, 对比度更明显。

核的生成过程为:

先创建全负值矩阵, 再在中心点设置强正权重形成对比, 最后归一化。

生成代码如下:

```
1     def high_detail_kernel(size=5, center_gain=24, surround_weight=-1):
2         kernel = np.zeros((size, size))
3         c = size // 2
4
5         kernel.fill(surround_weight)
6         kernel[c-1:c+2, c-1:c+2] = 0
7         kernel[c, c] = center_gain
8
9         if np.sum(kernel) != 0:
10             kernel = kernel / np.sum(np.abs(kernel))
11     return kernel
```

- 高斯模糊核:

主要参数如下:

- 核尺寸 *size*
- 标准差 *sigma*: 控制模糊强度。值越大模糊效果越明显。

核的生成过程为:

先计算各点到中心的高斯距离权重, 再通过标准差 σ 控制衰减强度, 最后归一化。

生成代码如下:

```
1     def gaussian_kernel(size=5, sigma=1.0):
2         kernel = np.zeros((size, size))
3         c = size // 2
4
5         for x in range(size):
6             for y in range(size):
7                 dx, dy = x - c, y - c
8                 kernel[x,y] = np.exp(-(dx**2 + dy**2)/(2*sigma**2))
9     return kernel / np.sum(kernel)
```

3.4.4 运行结果与分析

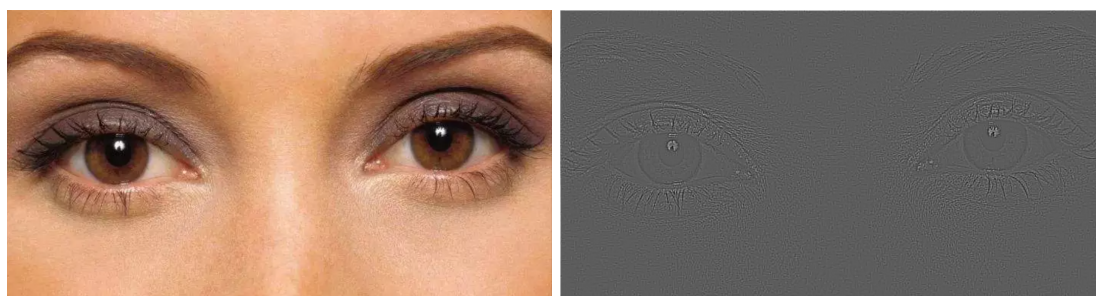


Figure 18: 增强中心核处理效果对比（原图左，处理结果右）

在原图的虹膜纹理边缘，即“中心对称性”强的地方，增强中心核能有效地识别提取；在睫毛根部处，对比度有一定的提升，可以较明显的分辨出睫毛位置；在其他平滑处呈现灰色，说明对无用信息可以有效忽略。但是，在平坦皮肤区噪声标准差有一定的提升，会干扰最终的识别效果。

整体来说，该核基本符合设计目标，在中心对称敏感型任务（如生物特征识别）中表现突出，但需权衡噪声放大效应。对于需要保留低频信息的场景，可以与平滑核级联使用。

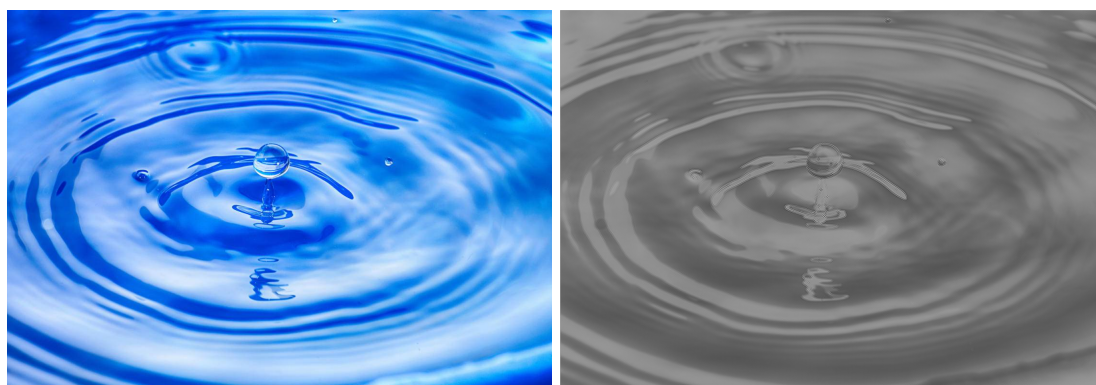


Figure 19: 波纹检测核处理效果对比（原图左，处理结果右）

在原图的水面波纹处，呈现出明显的波纹响应，本质是由于该波纹检测核通过中心峰值权重与对角线正负交替权重，对水滴落点处的同心圆波纹产生特异性响应，并且核的对角线权重与波纹扩散方向匹配，形成共振增强，故周期性纹理具有较强的敏感性。在溅射水珠部分，对比度有一定的提升，但由于水珠的纹理方向与核的方向不匹配，响应较弱。对于背景水面的反射，其噪声标准差上升程度并不高，不影响整体图像的平滑度。

整体来说，该核在流体动态分析中，对同心圆波纹特异性强、对水花溅射能精准定位、抗干扰能力优于传统算子。但对非规则波纹的响应强度不高，可以配合其他方向特异核使用。

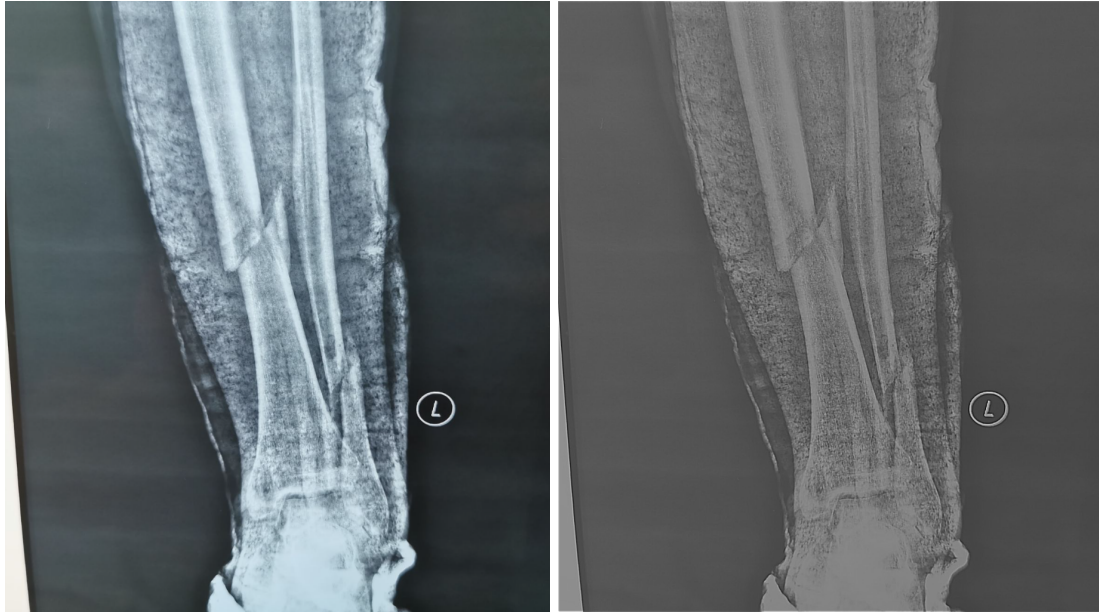


Figure 20: 高频细节核处理效果对比（原图左，处理结果右）

该高频细节核通过中心增强与周边抑制的设计，能够提升原图中模糊的骨折线清晰度。对于肌肉组织处，其灰度梯度变化不大，能被一定程度平滑处理，使得骨骼轮廓进一步突出，但其噪声较高。并且对于非骨折区域，可能会出现伪影，影响对于骨折区域的判断。

整体来说，该核对于骨折线等高频细节提取效果较好，符合预期；但对平滑区域噪声放大较明显，需要进一步对参数进行处理并同一些平滑核级联使用。



Figure 21: 高斯模糊核处理效果对比（原图左，处理结果右）

该高斯模糊核有效降低背景立体字边缘锐度、发丝和发卡轮廓清晰度，使得高频细节消除效果好，模糊程度高。在背景虚化、风格转换和信息分级场景有应用价值。但其色彩信息丢失严重，可以在 RGB 三个通道分别进行高斯模糊，最大程度的保留原图色彩信息。

整体来说，该核在图像平滑和风格化处理上表现良好，模糊程度高，基本达成设计目标。

4 综合讨论

4.1 人工设计卷积核与训练卷积核的对比分析

人工设计的卷积核和训练得到的卷积核均表现出方向敏感性，而在生物视觉系统的初级视觉皮层中，特定神经元也仅对特定方向的边缘或纹理产生强烈响应。这印证了生物视觉与人工模型

的趋同性，也表明这种方向选择性是视觉信息分层处理的基础。

在**可解释性与适应性**方面，人工核具有明确的数学定义和物理意义，适用于小样本或无需训练的场景（如图像预处理）；而训练核通过反向传播优化，能自适应复杂任务（如自然图像分类、医学影像分割），但可解释性较差。因此可以通过混合架构设计，利用人工核引导训练、动态核切换、对训练核进行方向聚类分析来增强整个系统的可解释性与鲁棒性。

4.2 三维卷积在医学影像中的应用潜力

传统二维卷积在处理医学影像时存在显著局限——层间信息丢失。以肺部 CT 为例，单个切片仅能提供横断面信息，而进一步判断病情需综合分析其在连续切片中的形态演变。三维卷积通过**多模态数据融合**，直接处理体数据，保证空间连续性，动态提取丰富的特征信息。

近年来，人们开始意识到三维卷积的重要性，逐渐尝试在肺结节检测、脑卒中病灶分割、骨科手术导航等方面，利用级联 3D 卷积网络，辅助医疗诊断。

目前三维卷积面临计算复杂度高、医学标注成本高等挑战。将来可以利用稀疏采样和分离卷积等手段，降低资源消耗；引入自监督预训练提升模型泛化性；通过神经架构搜索与软硬件协同设计，提高其实时性，激发 3D 卷积在医学影像分析中的应用潜力。

4.3 循环展开法的优化策略

循环展开法是提升卷积计算效率的重要技术，其核心思想是将滑动窗口操作转换为矩阵乘法运算。然而该算法面临显著内存挑战，容易导致显存溢出。例如 $128 \times 128 \times 128$ 的 CT 扫描，（batch=4）展开后显存需求可达 32GB，远超消费级 GPU 容量。此外，90% 时间在数据搬运、并行度受限、缓存命中率低等问题。

针对上述问题，可以通过以下策略进行优化：

- **分块计算策略**：将输入分割为子区域逐块处理，利用双缓冲技术隐藏数据传输延迟，牺牲部分并行性换取内存节省。
- **稀疏化展开**：基于输入特征图的特征幅值进行动态掩码生成，压缩稀疏行存储，并利用稀疏矩阵乘法优化。
- **加速计算方法**：针对 GPU 的 Tensor Core 优化矩阵惩罚的数据排布，提升计算单元利用率；利用 Winograd 快速卷积、混合精度计算等手段，提高计算效率。
- **核分解**：将大卷积核分解为多个小核，减少单次展开的参数数量，降低内存需求。

4.4 自定义卷积核设计的创新方法

自定义卷积核可以结合人工先验核与可训练核的混合架构，进行跨模态卷积核设计、动态三维卷积、轻量化部署。此外，在医学临床应用方面，自定义卷积核设计有如下创新方法：

- **基于解剖学先验的核设计范式**：采用问题驱动 → 数学建模 → 生物验证的创新路径，设计心脏运动专用核、肺血管增强核等多尺度融合设计。
- **动态可调核技术**：突破传统核固定参数设计限制，引入参数自适应机制，增加空间通道双路注意力。

- **物理约束核设计:** 结合物理模型与数据驱动方法, 设计具有物理约束的核, 如基于流体力学的波纹检测核、基于光学成像的高频细节核等。
- **核架构搜索优化:** 进行自动化流程设计, 进化搜索策略, 微分架构搜索。

4.5 总结与展望

本实验通过手动实现二维卷积操作, 并拓展到一维及三维卷积, 深入探究了卷积的数学原理及其在深度学习中的应用。在基础任务中, 我们验证了卷积操作的正确性、卷积核的可解释性与特征提取能力, 并通过优化训练策略提升了 Lenet 模型性能; 进阶任务则展示了三维卷积在视频处理中的潜力以及人工设计卷积核的灵活性。

实验结果表明, 卷积操作在图像、文本、视频等多模态数据中均具备强大的特征表达能力。未来工作可进一步探索计算效率优化、生物启发式卷积设计以及三维卷积在医学影像等领域的应用, 以推动更高效、更可解释的卷积神经网络发展。

5 成员分工

Table 7: 分工安排详表

任务	王肖鉴	冯越	方俊皓	梁启宸
	@WhXcjm	@Fay-777	@funjunhao	@raregas-48
Coding 部分				
B1. Conv2d - core code with vectorize			1.2	
B1. Conv2d - test with common kernels			0.3	0.5
B1. Conv2d - validity test & benchmark			1	
B2. LeNet-mnist - core code		2		
B2. LeNet-mnist - accuracy		1		
B2. LeNet-mnist - kernel demonstrate	0.3	0.9		
A1. Conv1d - core code		0.8		
A1. Conv1d - accuracy		0.3		
A1. Conv1d - custom kernel		0.3		
A2. Conv3d - core code	1.2			
A2. Conv3d - custom kernel	1			
A3. Benchmark - Accuracy	1			
A3. Benchmark - Performance	1.5			
A4. custom kernels (2D)				2.4

Continued on next page

Table 7: 分工安排详表 (Continued)

Slides 部分				
Basic1: what is conv2d			0.5	
Basic1: padding & stride			0.5	
Basic1: vectorization	0.2		0.3	
Basic1: benchmark			0.3	
Basic1: custom kernels				0.5
Basic2: lenet	0.1	0.1		0.5
Conv1d	0.3	0.1		
Conv3d	0.5			
Benchmark (conv1d & conv3d)	0.5			
Integration	0.5			
Prezentation	1	1	1	0.7
Report 部分				
1 Intro		0.4		
2.1 conv2d	0.4		2	
2.1.4 conv2d custom kernels	0.1			0.5
2.2 lenet		2		
3.1 conv1d		0.4		
3.2 conv3d & its custom kernels	1.5			
3.3 benchmark	1.2			
3.4 customKernels				1.5
4 Discussion				0.6
5 Arrangement	0.4			
Integration	1.5			
项目开展及最终提交				
requirement.txt	0.3		0.3	
Arrangement & Summary	1	0.2		
Readme.md	0.5	0.5	0.5	0.5
大模型使用说明		0.3		
合计工作量	15	10.3	7.9	7.7
工作量占比	36.67%	25.18%	19.32%	18.83%

6 大模型使用说明

在项目开发过程中，我们在多个环节利用了大模型（包括 **GitHub Copilot**、**腾讯元宝**等）进行辅助，具体应用场景如下：

1. **Report 的 $\text{\LaTeX}$ 框架搭建**：借助大模型获取基础模板结构建议，获得章节划分、标题格式、段落排版等基础元素的框架内容，并在此基础上针对性调整和完善，以适配本项目报告需求。
2. **代码画图辅助**：利用大模型生成初始绘图代码框架，包括基本绘图指令、图层设置、元素绘制顺序等代码片段，随后根据实际需求进行参数调整和细节完善。
3. **CNN 向量化操作参考**：在实现 CNN 层的向量化计算逻辑时，参考大模型提供的理论原理阐释和常见算法思路，辅助理解并优化 CNN 向量化操作。
4. **依赖文件生成**：依赖大模型推荐的 `pipreqs` 工具自动生成 `requirements.txt`，准确识别并导出项目所需的全部 Python 依赖库。
5. **性能测试方案设计**：参考大模型的建议设计性能测试方案，确保测试流程科学合理且覆盖全面。
6. **动图处理与图片合并工具开发**：借助大模型的方法处理动图，在设计图片合并布局后，利用大模型生成合并动图的工具代码。
7. **自定义卷积核设计**：基于大模型的分析能力，辅助针对具体任务调整核参数，并通过运行效果再调核参数，确保实现效果符合设计目的。

References

- [1] 机器学习算法那些事 【卷积神经网络】4. 卷积算子. Available: <https://mp.weixin.qq.com/s/h8CwE0AgvaML69CugBUqwa>
- [2] Zhang, A.; Lipton, Z. C.; Li, M.; Smola, A. J. *Dive into Deep Learning*. Cambridge University Press, 2021.
- [3] PaddleEdu. 卷积 (*Convolution*) . Available: https://paddlepedia.readthedocs.io/en/latest/tutorials/CNN/convolution_operator/Convolution.html
- [4] © 2025 Tencent. 腾讯元宝. Available: <https://yuanbao.tencent.com/>
- [5] © 2025 GitHub, Inc. *GitHub Copilot*. Available: <https://github.com/features/copilot>